THAKUR COLLEGE OF SCIENCE AND COMMERCE

Affiliated to University of Mumbai 2026 – 2027

PRACTICAL JOURNAL OF
APPLIED DATA SCIENCE

SUBJECT GUIDE
Ms. SHREYA MULEY
(Asst. Professor)

SUBMITTED BY:
VINAY SANJAY PANCHAL
ROLL NO: 264928

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR QUALIFYING M.Sc.DS PART I
(SEMESTER I EXAMINATION)

Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
AUTONOMOUS COLLEGE, PERMANENTLY AFFILIATED TO UNIVERSITY OF MUMBAI
NAAC Accredited Grade 'A' (3rd Cycle) & ISO 9001: 2015 (Certified)
Best College Award by University of Mumbai for the Year 2018-2019
CELEBRATING 25 YEARS OF GLORY

CERTIFICATE OF APPROVAL

This is to certify that **Mr. Vinay Sanjay Panchal** student of "Master of Science (DATA SCIENCE)" of "Thakur College of Science and Commerce". Roll No. **264928** has successfully completed and submitted the Practical & Assignment entitled "**Applied Data Science**" in the partial fulfillment as per the syllabus defined by the University of Mumbai in the academic year 2026-2027.

It is further certified that the student has completed all the required phases of the practical & assignment.

_____

HEAD OF DEPARTMENT

_____ _____

PROFESSOR INCHARGE **EXTERNAL EXAMINER**

Index

Sr. No.	Date	Aim	Pg No.	Sign
1		A. Write a Python Program to convert CSV to HORUS. B. Write a Python Program to convert Image to HORUS. C. Write a Python Program to convert Audio to HORUS.		
2		A. Write a Python Program to remove leading and trailing spaces. B. Write a Python Program to remove non-printable characters from a string. C. Write a Python Program to format date.		
3		A. Write a python program to implement Data Binning. B. Write a python program to implement averaging of data. C. Write a python program to implement Outlier Detection.		
4		A. Write a python program to implement logging and process monitoring. B. Write a python program to implement Data Provenance and Transformation Tracking.		
5		A. Write a python program to handle missing values with Mean Imputation. B. Write a python program to handle outliers using Z-score method.		
6		A. Write a python program to implement normalization using Min-Max Scaling. B. Write a python program to implement feature transformation.		
7		A. Write a python program to perform one Sample t-Test. B. Write a python program to perform two Sample t-Test.		
8		A. Write a python program to implement Logistic Regression. B. Write a python program to implement K-Means Clustering.		
9		A. Write a python program to implement Principal Component Analysis. B. Write a python program to implement Support Vector Machine.		

Practical No: 1A

Aim: Write a Python Program to convert CSV to HORUS

Code:

```
#P.1.A CSV to HORUS
import pandas as pd

# Input File
input_file = "/content/Country_Codes.csv"

# Read CSV file
df = pd.read_csv(input_file, encoding="latin1")

print("========== Input Data ==========")
print(df)
print("================================")

# Data Processing
processed_df = df.copy()

# Remove unwanted columns
processed_df.drop(columns=["ISO-2-CODE", "ISO-3-Code"], inplace=True)

# Rename columns
processed_df.rename(columns={
    "Country": "CountryName",
    "ISO-M49": "CountryNumber"
}, inplace=True)

# Set CountryNumber as index
processed_df.set_index("CountryNumber", inplace=True)

# Sort by CountryName in descending order
processed_df.sort_values(by="CountryName", ascending=False, inplace=True)

print("\n========== Processed Data ==========")
print(processed_df)
print("====================================")

# Save Output
output_file = "output.csv"
processed_df.to_csv(output_file)

print(f"\nConversion completed successfully!")
print(f"Output saved as: {output_file}")
```

Input File:

Country_Codes.csv

1 to 10 of 247 entries Filter

Country	ISO-2-CODE	ISO-3-Code	ISO-M49
Afghanistan	AF	AFG	4
Aland Islands	AX	ALA	248
Albania	AL	ALB	8
Algeria	DZ	DZA	12
American Samoa	AS	ASM	16
Andorra	AD	AND	20
Angola	AO	AGO	24
Anguilla	AI	AIA	660
Antarctica	AQ	ATA	10
Antigua and Barbuda	AG	ATG	28

Output:

```
264928_VINAY SANJAY PANCHAL
Choose Files Country_Codes.csv
Country_Codes.csv(text/csv) - 5901 bytes, last modified: 7/3/2026 - 100% done
Saving Country_Codes.csv to Country_Codes.csv
=========== Original Data =============
                        Country ISO-2-CODE ISO-3-Code  ISO-M49
0                   Afghanistan         AF         AFG         4
1                 Aland Islands         AX         ALA       248
2                       Albania         AL         ALB         8
3                       Algeria         DZ         DZA        12
4                  American Samoa         AS         ASM        16
..                            ...        ...         ...       ...
242  Wallis and Futuna Islands         WF         WLF       876
243               Western Sahara         EH         ESH       732
244                        Yemen         YE         YEM       887
245                       Zambia         ZM         ZMB       894
246                     Zimbabwe         ZW         ZWE       716

[247 rows x 4 columns]
=========================================

=========== Processed Data =============
                               CountryName
CountryNumber
716                              Zimbabwe
894                                Zambia
887                                 Yemen
732                         Western Sahara
876             Wallis and Futuna Islands
...                                   ...
16                         American Samoa
12                                Algeria
8                                 Albania
248                         Aland Islands
4                             Afghanistan

[247 rows x 1 columns]
=========================================

Conversion Completed Successfully!
Output saved as: output.csv
```

Output File:

output.csv

1 to 10 of 247 entries Filter

CountryNumber	CountryName
716	Zimbabwe
894	Zambia
887	Yemen
732	Western Sahara
876	Wallis and Futuna Islands
850	Virgin Islands, US
704	Viet Nam
862	Venezuelaÿ(Bolivarian Republic)
548	Vanuatu
860	Uzbekistan

Theory:

Data transformation is an important process in data preprocessing that involves converting raw data into a structured and meaningful format suitable for analysis, reporting, or integration with other systems. Raw datasets often contain unnecessary information, inconsistent column names, missing values, or unordered records that reduce data quality and make analysis difficult. Data transformation techniques help improve the accuracy, consistency, and usability of the dataset by organizing it into a standardized format.

The CSV (Comma-Separated Values) file format is one of the most widely used formats for storing and exchanging tabular data. It is lightweight, easy to create, and compatible with various software applications such as spreadsheet programs, databases, and programming languages. However, before CSV data can be used for analysis or imported into another application, it often requires preprocessing to remove unnecessary information, rename columns, reorder records, and organize the data according to specific requirements.

Python is a popular programming language for data processing because of its simplicity and extensive collection of libraries. One of the most powerful libraries for handling tabular data is Pandas. Pandas provides the **DataFrame**, a two-dimensional data structure that stores data in rows and columns, making it easy to manipulate datasets. The **read_csv()** function is used to load the CSV file into a **DataFrame**, allowing various data transformation operations to be performed efficiently. The encoding parameter ensures that special characters in the input file are read correctly without causing errors.

To protect the original dataset, a copy of the **DataFrame** is created before making any modifications. This is considered a good programming practice because it preserves the source data and prevents accidental changes. The program then removes unwanted columns, such as ISO-2-CODE and ISO-3-Code, using the **drop()** function. Eliminating unnecessary columns simplifies the dataset, reduces redundancy, and improves readability.

The remaining columns are renamed using the **rename()** function to provide standardized and meaningful names such as CountryName and CountryNumber. Standardized naming conventions make the dataset easier to understand and ensure compatibility with the required HORUS format. The CountryNumber column is then assigned as the **DataFrame** index using the **set_index()** function. Setting an index provides a unique identifier for each record and enables efficient data retrieval and organization.

The records are further arranged in descending order based on the CountryName column using the **sort_values()** function. Sorting the data improves presentation, makes searching easier, and ensures that the output follows a consistent order. After completing all transformation steps, the processed DataFrame is exported to a new CSV file using the **to_csv()** function.

Overall, this program demonstrates how Python and the Pandas library can automate data preprocessing tasks such as data loading, column removal, renaming, indexing, sorting, and exporting. Automation reduces manual effort, minimizes human errors, and ensures consistency across large datasets. These techniques are fundamental in data analytics, business intelligence, database management, and software development, where clean, organized, and reliable data is essential for accurate analysis and effective decision-making.

Practical No: 1B

Aim: Write a Python Program to convert Image to HORUS

Code:

```
# P.1.B IMAGE to HORUS

from skimage import io
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Input Agreement
sInputFile = r"/content/Image.png"
data = io.imread(sInputFile)

plt.imshow(data)
plt.title("Input Image")
plt.axis('off')
plt.show()

print('Input Data')
print('Height (X):', data.shape[0])
print('Width  (Y):', data.shape[1])
print('Channels  :', data.shape[2])

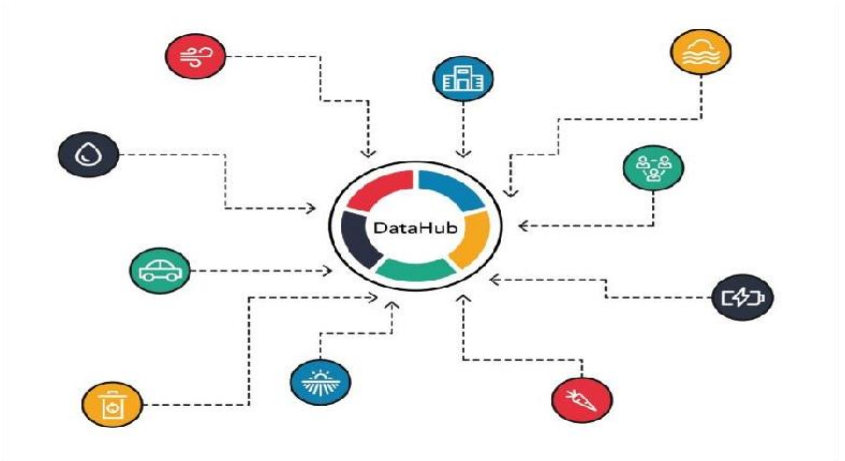
# Processing Rules
height, width, _ = data.shape
rows = []

for x in range(height):
    for y in range(width):
        r, g, b, a = data[x, y]
        rows.append([x, y, r, g, b, a])

process = pd.DataFrame(rows, columns=['XAxis', 'YAxis', 'Red', 'Green', 'Blue', 'Alpha'])
print(process.head())
print('Rows: ', process.shape[0])
print('Columns: ', process.shape[1])

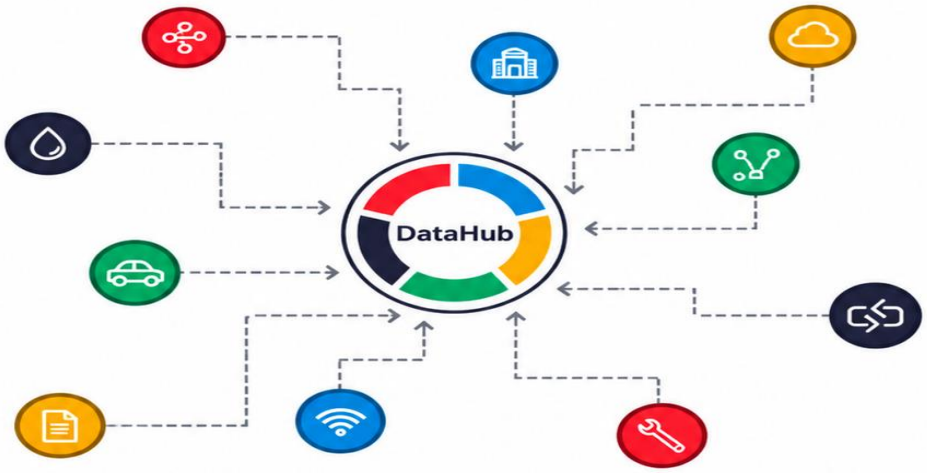
# Output Agreement
sOutputFile = "HORUS-IMG.csv"
process.to_csv(sOutputFile, index=False)
print('Image to HORUS - Done')
```

Input File:



Output:

```
    264928_VINAY SANJAY PANCHAL
    Saving Image.png to Image (1).png
***
                                Input Image
```




```
    Input Data
    Height (x): 1553
    Width (y): 2100
    Channels: 4

    First 5 Rows:
       XAxis  YAxis   Red   Green   Blue   Alpha
    0      0      0   255     255    255     255
    1      0      1   255     255    255     255
    2      0      3   255     255    255     255
    3      0      3   255     255    255     255
    4      0      4   255     255    255     255

    Rows: 3213900
    Columns: 6

    Image to HORUS Done!
    264928_VINAY SANJAY PANCHAL
```

Output File:

	A	B	C	D	E	F
1	XAxis	YAxis	Red	Green	Blue	Alpha
2	0	0	255	255	255	255
3	0	1	255	255	255	255
4	0	2	255	255	255	255
5	0	3	255	255	255	255
6	0	4	255	255	255	255
7	0	5	255	255	255	255
8	0	6	255	255	255	255
9	0	7	255	255	255	255
10	0	8	255	255	255	255
11	0	9	255	255	255	255
12	0	10	255	255	255	255

Theory:

The “Image to HORUS” practical demonstrates the process of converting a digital image into a structured numerical dataset. A digital image is composed of thousands (or millions) of tiny picture elements called pixels. Each pixel stores color information in the form of Red (R), Green (G), Blue (B) values, and in some cases an additional Alpha (A**)** value representing transparency. By extracting these pixel values, an image can be transformed into a tabular format that is suitable for data analysis, machine learning, and image processing tasks.

In this practical, the image is first uploaded using the Google Colab environment. The scikit-image (skimage) library is used to read the image file, while Matplotlib is used to display the uploaded image. The program then determines the image dimensions, including its height (rows), width (columns), and the number of color channels. Images may be of three types:

- **Grayscale** – contains only intensity values.
- **RGB** – contains Red, Green, and Blue color channels.
- **RGBA** – contains Red, Green, Blue, and Alpha (transparency) channels.

The program scans the image pixel by pixel using nested loops. For every pixel, it records the X-axis position (row), Y-axis position (column), and the corresponding Red, Green, Blue, and Alpha (RGBA) values. If the image is grayscale, the same intensity value is assigned to the R, G, and B channels, while the Alpha value is set to 255 (fully opaque). Similarly, if the image is RGB without an Alpha channel, the Alpha value is also assigned 255.

After extracting all pixel information, the data is stored in a Pandas DataFrame, which organizes the information into six columns:

- **XAxis** – Row position of the pixel
- **YAxis** – Column position of the pixel
- **Red** – Red color intensity (0–255)
- **Green** – Green color intensity (0–255)
- **Blue** – Blue color intensity (0–255)
- **Alpha** – Transparency value (0–255)

Finally, the DataFrame is exported as a CSV (Comma-Separated Values) file named HORUS-IMG.csv. This CSV file represents the complete image in numerical form and can be used for further processing such as feature extraction, computer vision, image reconstruction, data mining, and artificial intelligence applications.

This practical highlight the concept that an image is not only a visual object but also a collection of numerical data. Converting images into structured datasets enables efficient storage, analysis, and processing using data science and machine learning techniques. Hence, the Image to HORUS conversion serves as an important preprocessing step in many image analysis and computer vision applications.

Practical No: 1C

Aim: Write a Python Program to convert Audio to HORUS

Code:

```
# P.1.C AUDIO to HORUS

from scipy.io import wavfile
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Input
file = r"/content/Audio.wav"
print('Processing:', file)

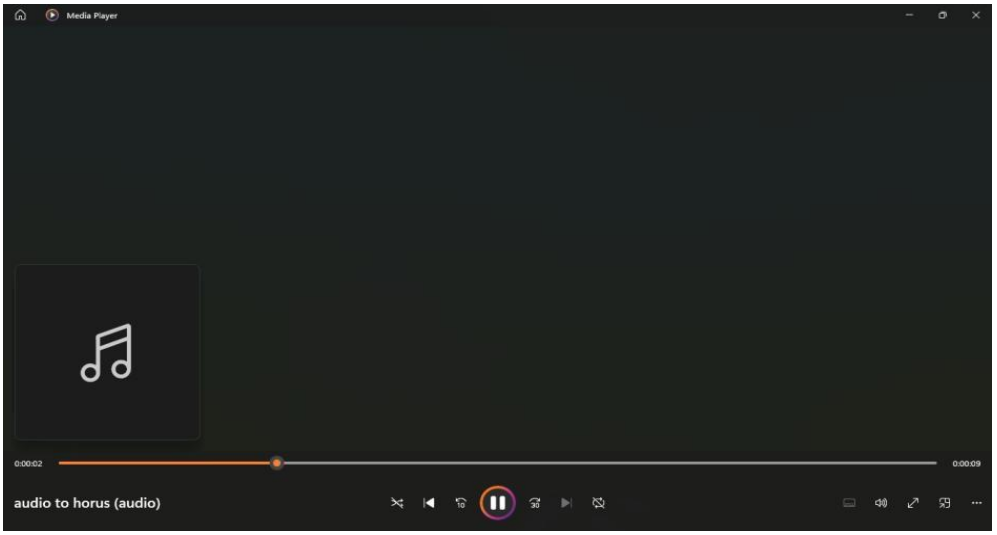
InputRate, InputData = wavfile.read(file)

# Processing
if InputData.ndim == 2 and InputData.shape[1] == 2:
    ProcessData = pd.DataFrame(InputData, columns=['Ch1', 'Ch2'])
else:
    ProcessData = pd.DataFrame(InputData, columns=['Ch1'])

print("First few records (HORUS Audio Format):")
print(ProcessData.head())

# Output
OutputFile = r"Audio_HORUS_Output.csv"
ProcessData.to_csv(OutputFile, index=False)
print('Audio to HORUS - Done')
```

Input File:



Output:

```
264928_VINAY SANJAY PANCHAL
Saving audio to horus (audio).wav - 2040406 bytes, last modified: 7/3/2026 - 100% done
Saving audio to horus (audio).wav to audio to horus (audio).wav
Processing: audio to horus (audio).wav
Sample Rate: 44100 Hz
Number of Samples: 510072
Channels: 2

First few records (HORUS Audio Format):
   Ch1  Ch2
0    0    0
1    0    0
2    0    0
3    0    0
4    0    0

Audio to HORUS Done!
264928_VINAY SANJAY PANCHAL_Output.csv
```

Output File:

	A	B
4670	Ch1	Ch2
4671	1393	-10005
4672	906	-10006
4673	435	-9870
4674	82	-9627
4675	-94	-9360
4676	-116	-9155
4677	-45	-9058
4678	87	-9064
4679	321	-9160
4680	774	-9353
4681	1564	-9655
4682	2682	-10039
4683	3966	-10427
4684	5234	-10729

Theory:

The "Audio to HORUS" practical demonstrates the process of converting a digital audio signal into a structured dataset. A digital audio file is made up of thousands of sound samples recorded at regular time intervals. Each sample represents the amplitude (strength) of the sound wave at a particular instant. By extracting these samples and storing them in a tabular format, the audio data becomes suitable for analysis, visualization, machine learning, and digital signal processing applications.

In this practical, a WAV (Waveform Audio File Format) file is uploaded using the Google Colab environment. The SciPy library, specifically the **`wavfile`** module, is used to read the audio file. The program extracts two important pieces of information: the sample rate and the audio samples. The sample rate indicates the number of samples recorded per second and is measured in Hertz (Hz**)**. For example, a sample rate of 44,100 Hz means that 44,100 sound samples are recorded every second, providing high-quality audio reproduction.

The program also identifies whether the uploaded audio is mono or stereo. A mono audio file contains a single channel of sound, while a stereo audio file contains two channels, namely the left channel (Ch1) and the right channel (Ch2). If the audio is stereo, both channels are stored separately in the dataset. If the audio is mono, only one channel is stored.

The extracted audio samples are then organized into a Pandas DataFrame, which converts the audio signal into a structured table. The columns in the DataFrame represent the audio channels:

- **Ch1** – First audio channel (or the only channel in mono audio)
- **Ch2** – Second audio channel (available only for stereo audio)

Each row of the DataFrame corresponds to a single audio sample captured at a specific point in time. The program displays the first few records to verify that the conversion has been performed correctly. Finally, the DataFrame is exported as a CSV (Comma-Separated Values) file named Audio_HORUS_Output.csv, which represents the complete audio signal in numerical form.

The generated CSV file can be used for various applications such as audio signal analysis, speech recognition, sound classification, noise detection, music analysis, feature extraction, and machine learning. Converting audio into a structured dataset enables researchers and data scientists to process sound data using statistical and analytical techniques instead of working directly with raw audio files.

Thus, the "Audio to HORUS" practical illustrates how digital audio signals can be transformed into structured numerical data, making them easier to analyze, visualize, and use in data science, artificial intelligence, and digital signal processing applications.

Practical No: 2A

Aim: Write a program for Removing Leading & Trailing Spaces.

Code:

```
# P.2.A Removing Leading & trailing Spaces
print("264928_VINAY SANJAY PANCHAL")
data = '11 Data Science !!!'
print('>', data, '<')
cleandata = data.strip()
print('>', cleandata, '<')
```

Output:

```
264928_VINAY SANJAY PANCHAL
> 11 Data Science !!!!
> 11 Data Science !!!!
```

Theory:

Data cleaning is one of the most important steps in data preprocessing. Before performing data analysis, machine learning, or database operations, it is essential to remove unwanted characters and spaces from the input data. Extra spaces at the beginning or end of a string may not be visible to the user, but they can create problems during searching, sorting, comparison, and storage of data. Python provides several built-in string methods for cleaning text, among which the **strip()** method is one of the most commonly used.

The **strip()** method removes all leading (left-side) and trailing (right-side) whitespace characters from a string. These whitespace characters include spaces, tabs (\t), newline characters (\n), and other blank characters. It does not remove spaces that exist between words because those spaces are considered part of the actual content of the string. The syntax of the method is:

When the **strip()** method is applied, it returns a new string with the unwanted spaces removed, while the original string remains unchanged since Python strings are immutable. Therefore, the cleaned string is usually stored in another variable for further use.

In this practical, a sample string containing unnecessary spaces before and after the text is created. The original string is displayed by enclosing it between special symbols (> and <) so that the extra spaces become clearly visible. The **strip()** method is then applied to remove these unwanted spaces. The cleaned string is stored in a new variable and printed again. The output shows that only the leading and trailing spaces have been removed, while the spaces between the words remain unchanged.

The **strip()** method plays a significant role in data preprocessing and text processing. Data collected from user input forms, CSV files, Excel sheets, databases, APIs, and web scraping often contains accidental spaces. If these spaces are not removed, the same value may be treated as different. For example, "Data Science" and " Data Science " appear identical to humans but are considered different strings by a computer. This can lead to duplicate records, incorrect filtering, failed searches, and inaccurate analysis. By using **strip()**, the data becomes clean, standardized, and ready for processing.

The **strip()** method is also widely used in applications such as form validation, natural language processing (NLP), data mining, machine learning, database management, and file handling. It improves data consistency, increases processing accuracy, and reduces errors caused by unwanted whitespace characters. Python also provides related methods such as **lstrip(),** which removes spaces only from the left side, and **rstrip(),** which removes spaces only from the right side of a string.

Thus, the **strip()** method is an essential Python string function used for removing unnecessary leading and trailing spaces from text data. It is a simple yet powerful preprocessing technique that ensures clean, accurate, and standardized data, making it suitable for data analysis, machine learning, and other data science applications.

Practical No: 2B

Aim: Write a program for Removing Non-Printable String.

Code:

```
# P.2.B Removing Non-Printable String
print("264928_VINAY SANJAY PANCHAL')
import string
data = "Data\x00Science and\x02 ML is \x10fun!!!"
clean = "".join(filter (lambda x: x in string.printable, data))
print(clean)
```

Output:

```
264928_VINAY SANJAY PANCHAL
DataScience and ML is fun!!!!
```

Theory:

Data preprocessing is a crucial step in data science, machine learning, and text analytics. Real-world data collected from files, databases, web pages, APIs, sensors, or communication systems often contains unwanted or hidden characters known as non-printable characters. These characters are not visible when the text is displayed on the screen, but they are stored within the data and can cause errors during processing, searching, comparison, file handling, and database storage. Therefore, removing non-printable characters is an important data-cleaning technique.

In Python, the string module provides a predefined constant called **string.printable**, which contains all printable ASCII characters such as uppercase letters, lowercase letters, digits, punctuation marks, and whitespace characters (space, tab, newline, etc.). Any character that is not present in **string.printable** is considered a non-printable character.

In this practical, a sample string is created containing several hidden control characters such as \x00 (Null), \x02 (Start of Text), and \x10 (Data Link Escape). These characters are represented using hexadecimal escape sequences and cannot be seen in the normal text output. Although invisible, they are still part of the string and may affect data processing.

The program uses the **filter()** function along with a lambda function to examine each character in the string. The lambda function checks whether each character belongs to **string.printable**. If the character is printable, it is retained; otherwise, it is discarded. The remaining printable characters are then combined using the **join()** method to form a clean string without any hidden or unwanted characters.

Removing non-printable characters is widely used in text preprocessing, Natural Language Processing (NLP), data mining, machine learning, web scraping, log file analysis, database management, and file processing. It improves data quality, prevents encoding issues, reduces processing errors, and ensures that only meaningful and readable characters remain in the dataset. This cleaning process is especially useful when handling text collected from multiple sources where hidden control characters may be introduced during transmission or storage.

Thus, this practical demonstrates an efficient method of cleaning textual data by removing non-printable characters using Python's **string.printable**, **filter(),** lambda, and **join()** functions. It ensures that the resulting text is clean, readable, standardized, and suitable for further analysis and processing in data science applications.

Practical No: 2C

Aim: Write a Python Program for Formatting Date

Code:

```
#P.2.C Formatting Date
print "264928_VINAY SANJAY PANCHAL"
import datetime as dt
date = dt.date(2026, 7, 8)
date = format(date, '%Y-%m-%d')
print(date)
fdate = dt.datetime.strptime(date, '%Y-%m-%d')
fdate = format (fdate, '%d %B %Y')
print(fdate)
```

```
264928_VINAY SANJAY PANCHAL
2026-07-08
08 July 2026
```

Theory:

Date and time handling is an essential part of programming and data science. Many applications such as banking systems, attendance management, e-commerce, healthcare, data analytics, and machine learning require dates to be stored, processed, and displayed in different formats. Since users and systems often use different date formats, Python provides the datetime module to create, manipulate, convert, and format date and time values efficiently.

In this practical, the datetime module is imported using the alias dt. First, a date object is created by specifying the year, month, and day. The date is then converted into a standard string format using the **format()** function with the format specifier **%Y-%m-%d**, where **%Y** represents the four-digit year, **%m** represents the two-digit month, and **%d** represents the two-digit day. This format is commonly used in databases, software applications, and international data exchange because it follows the ISO 8601 standard.

After converting the date into a string, the program uses the **strptime()** method to convert the formatted string back into a datetime object. This process is known as parsing. Once the string has been converted into a datetime object, it is reformatted into a more user-friendly format using the **format()** function with the format specifier **%d %B %Y**. In this format**, %d** displays the day**, %B** displays the full month name, and **%Y** displays the four-digit year. For example, the date 2026-07-08 is displayed as 08 July 2026, which is easier for users to read and understand.

Some commonly used date format specifiers include:

%Y – Four-digit year (e.g., 2026)

%m – Two-digit month (01–12)

%d – Two-digit day (01–31)

%B – Full month name (e.g., July)

%b – Short month name (e.g., Jul)

%A – Full weekday name (e.g., Wednesday)

Date formatting is widely used in data preprocessing, report generation, database management, web applications, file naming, logging systems, scheduling software, and business analytics. Standardizing dates helps maintain consistency across different systems, improves data quality, simplifies sorting and filtering operations, and avoids errors caused by different regional date formats.

Thus, this practical demonstrates how Python's datetime module can be used to create a date, convert it into different formats, parse date strings, and display dates in a readable format. Date formatting is an important skill in programming and data science, ensuring that date information is stored, processed, and presented accurately according to application requirements.

Practical No: 3A

Aim: Write a Python Program for Data Binning

Code:

```
# P.3.A: Data Binning

import pandas as pd
import matplotlib.pyplot as plt

# File path
sInputFile = "/content/IP_Data_Core.csv"

# Load dataset
df = pd.read_csv(sInputFile)

print("First 5 Records")
print(df.head())

# Equal Width Binning
df["Latitude_Bin"] = pd.cut(
    df["Latitude"],
    bins=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

print("\nBin Counts:")
print(df["Latitude_Bin"].value_counts().sort_index())

# Graph
df["Latitude_Bin"].value_counts().sort_index().plot(
    kind="bar",
    figsize=(8, 5),
    edgecolor="black",
    color="skyblue"
)

plt.title("Equal Width Binning of Latitude")
plt.xlabel("Latitude Bins")
plt.ylabel("Frequency")
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
```

Input File:

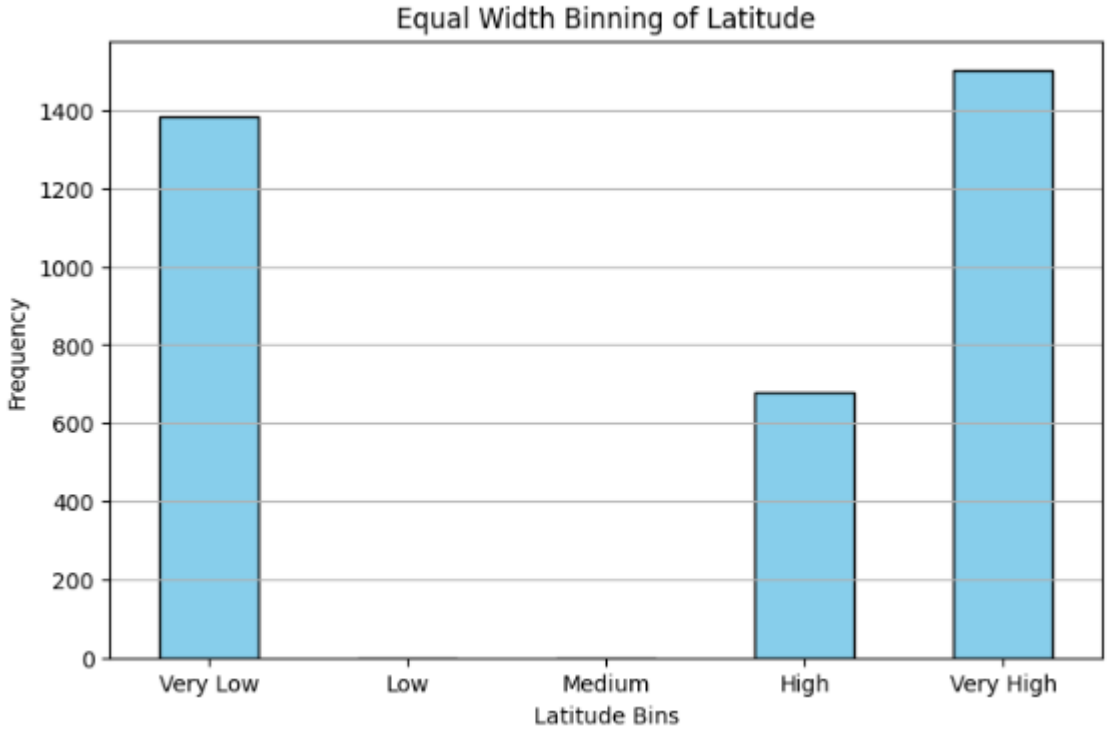
	A	B	C	D	E	F	G	H	I
1	ID	Country	Place Nam	Post Code	Latitude	Longitude	First IP Nu	Last IP Number	
2	1	US	New York	10017	40.7528	-73.9725	2.04E+08	2.04E+08	
3	2	US	New York	10017	40.7528	-73.9725	3.02E+08	3.02E+08	
4	3	US	New York	10017	40.7528	-73.9725	4.05E+08	4.05E+08	
5	4	US	New York	10017	40.7528	-73.9725	4.12E+08	4.12E+08	
6	5	US	New York	10017	40.7528	-73.9725	4.17E+08	4.17E+08	
7	6	US	New York	10017	40.7528	-73.9725	6.43E+08	6.43E+08	
8	7	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
9	8	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
10	9	US	New York	10017	40.7528	-73.9725	6.45E+08	6.45E+08	
11	10	US	New York	10017	40.7528	-73.9725	6.46E+08	6.46E+08	
12	11	US	New York	10017	40.7528	-73.9725	7.9E+08	7.9E+08	

Output:

```
264928_VINAY SANJAY PANCHAL
IP_Data_Core.csv
IP_Data_Core.csv(text/csv) - 213088 bytes, last modified: 7/26/2026 - 100% done
Saving IP_Data_Core.csv to IP_Data_Core (1).csv
First 5 Records:
   ID Country Place Name Post Code  Latitude  Longitude  First IP Number  \
0   1      US   New York     10017   40.7528   -73.9725        204276430
1   2      US   New York     10017   40.7528   -73.9725        301985791
2   3      US   New York     10017   40.7528   -73.9725        404678339
3   4      US   New York     10017   40.7528   -73.9725        411592954
4   5      US   New York     10017   40.7528   -73.9725        416784639

   Last IP Number
0       204276735
1       301985791
2       404678339
3       411592959
4       416784639

Bin Counts:
Latitude_Bin
Very Low     1383
Low             0
Medium          0
High          677
Very High    1502
Name: count, dtype: int64
```


Equal Width Binning of Latitude
Frequency
0
200
400
600
800
1000
1200
1400
Very Low
Low
Medium
High
Very High
Latitude Bins

Theory:

Data binning is a **data preprocessing and data reduction technique** used in data analytics, data mining, and machine learning. It involves grouping continuous numerical values into a fixed number of intervals called **bins**. Instead of analyzing every individual value, the data is divided into meaningful ranges, making it easier to understand patterns, trends, and distributions. Data binning helps simplify large datasets, reduce the effects of minor variations, and improve the efficiency of data analysis. It is commonly applied before performing statistical analysis, visualization, or machine learning tasks.

In this practical, the Python **Pandas** library is used to read a CSV dataset containing IP-related information, including a **Latitude** column. The dataset is loaded into a **DataFrame()** using the **`read_csv()`** function. Before performing binning, the program checks whether the **Latitude** column exists in the dataset. This validation prevents runtime errors if the required column is missing. The first five records of the dataset are displayed using the **`head()`** function to verify that the data has been loaded correctly.

Since datasets may contain missing or invalid values, the **Latitude** column is converted into a numeric format using the **`pd.to_numeric()`** function with the `errors='coerce'` option. Any non-numeric values are automatically converted into missing values. These missing values are then removed using the **`dropna()`** function. Cleaning the data before preprocessing is an important step because it improves the quality and reliability of the results obtained from data analysis.

The practical also includes **data visualization** using the **Matplotlib** library. A bar chart is created to represent the frequency of records in each latitude bin. The x-axis represents the latitude categories, while the y-axis shows the number of records present in each category. The graph uses a sky-blue color with black borders for better visibility. A title, axis labels, and grid lines are added to improve readability and make the chart easier to interpret. Visualization provides a quick understanding of the data distribution and helps users identify patterns without examining the raw numerical values.

Data binning has many practical applications in real-world scenarios. It is widely used in customer age classification, income grouping, weather analysis, geographical data processing, educational grading systems, medical research, and financial analysis. By grouping continuous data into meaningful intervals, organizations can make better decisions, simplify reports, and improve predictive models. It also reduces noise in datasets and enhances the performance of many machine learning algorithms.

In conclusion, this practical demonstrates the complete process of performing **Equal Width Data Binning** using Python and **Google Colab**. It covers loading a dataset, validating and cleaning data, converting values into equal-width categories, calculating the frequency of each bin, and visualizing the results using a bar chart. The practical highlights the importance of preprocessing in data analytics and shows how data binning transforms raw numerical information into a structured and meaningful form, making data analysis more efficient, accurate, and easier to understand.

Practical No: 3B

Aim: Write a Python Program for Averaging of Data

Code:

```
print("264928_VINAY SANJAY PANCHAL")

#P.3.B Averaging of Data
import pandas as pd
import matplotlib.pyplot as plt

# File path
sInputFile = "/content/IP_Data_Core.csv"

# Load dataset
df = pd.read_csv (sInputFile)
print("First 5 Records")
print(df.head())

# Group by Country
avg_data = df.groupby("Country") [["Latitude", "Longitude"]].mean()
print("\nAverage Latitude and Longitude")
print(avg_data)
```

Input File:

	A	B	C	D	E	F	G	H	I
1	ID	Country	Place Nam	Post Code	Latitude	Longitude	First IP Nu	Last IP Number	
2	1	US	New York	10017	40.7528	-73.9725	2.04E+08	2.04E+08	
3	2	US	New York	10017	40.7528	-73.9725	3.02E+08	3.02E+08	
4	3	US	New York	10017	40.7528	-73.9725	4.05E+08	4.05E+08	
5	4	US	New York	10017	40.7528	-73.9725	4.12E+08	4.12E+08	
6	5	US	New York	10017	40.7528	-73.9725	4.17E+08	4.17E+08	
7	6	US	New York	10017	40.7528	-73.9725	6.43E+08	6.43E+08	
8	7	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
9	8	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
10	9	US	New York	10017	40.7528	-73.9725	6.45E+08	6.45E+08	
11	10	US	New York	10017	40.7528	-73.9725	6.46E+08	6.46E+08	
12	11	US	New York	10017	40.7528	-73.9725	7.9E+08	7.9E+08	

Output:

```
264928_VINAY SANJAY PANCHAL
First 5 Records
   ID Country  Place Name  Post code   Latitude  Longitude  First IP Number
0   1       US    New York     10017   40.7528   -73.9725        204276480
1   2       US    New York     10017   40.7528   -73.9725        301985791
2   3       US    New York     10017   40.7528   -73.9725        404678339
3   4       US    New York     10017   40.7528   -73.9725        411592954
4   5       US    New York     10017   40.7528   -73.9725        416784639

Last IP Number
0    204276735
1    301985791
2    404678339
3    411592959
4    416784639

Average Latitude and Longitude
           Latitude  Longitude
Country
DE      48.143223    11.565514
GB      51.509406    -0.095591
US      40.747044   -73.986039
```

Theory:

Averaging of data is one of the most fundamental techniques used in **data analytics, statistics, and data science**. The average, also known as the **arithmetic mean**, represents the central value of a dataset and helps summarize a large amount of numerical information into a single meaningful value. Instead of analyzing every individual record, the average provides a simple way to understand the overall trend of the data. It is widely used in business analysis, geographical studies, scientific research, healthcare, finance, and machine learning to compare different groups and identify patterns in datasets.

In this practical, the **Pandas** library is used to read a CSV dataset named **IP_Data_Core.csv**. The dataset contains information related to IP addresses, including geographical details such as **Country**, **Latitude**, and **Longitude**. The dataset is loaded into a Pandas **DataFrame** using the `read_csv()` function. After loading the dataset, the first five records are displayed using the

`head()` function. Displaying these records helps verify that the dataset has been loaded successfully and allows the user to inspect the available columns before performing any analysis.

The main objective of this practical is to calculate the **average latitude and longitude for each country**. This is achieved using the **`groupby()`** function in Pandas. The `groupby("Country")` statement groups all records that belong to the same country into separate categories. Once the records are grouped, the **`mean()`** function is applied to calculate the arithmetic mean of the **Latitude** and **Longitude** columns for each country. The resulting table displays each country along with its average latitude and average longitude values.

The practical uses the **groupby()** method because it is one of the most powerful features of the Pandas library. It allows users to divide data into groups based on one or more columns and then perform operations such as average, sum, count, minimum, maximum, or standard deviation. In this case, the data is grouped according to the **Country** column, and the average values of **Latitude** and **Longitude** are calculated using the **`mean()`** function. This process is efficient even for datasets containing thousands of records.

Averaging of grouped data has many real-world applications. Businesses use averages to calculate average sales, average customer spending, and average profits. Educational institutions calculate average marks of students. Healthcare organizations analyze average patient statistics such as age, weight, or blood pressure. Weather departments calculate average temperatures and rainfall. In geographical information systems (GIS), average latitude and longitude values help identify the central location of regions or countries. Thus, averaging is an essential statistical technique used across multiple domains.

In conclusion, this practical demonstrates how to calculate the **average latitude and longitude for each country** using the Pandas library in Python. It involves loading a dataset, grouping records based on the **Country** column, and calculating the arithmetic mean using the `mean()` function. The practical illustrates the importance of data aggregation in simplifying complex datasets and generating meaningful insights. By summarizing geographical data into average values, analysts can better understand trends, compare different countries, and support informed decision-making in data science and analytics.

Practical No: 3C

Aim: Write a Python Program for Outlier Detection

Code:

```
print("264928_VINAY SANJAY PANCHAL")

#P.3.C Outlier Detection
import pandas as pd
import matplotlib.pyplot as plt

# File path
sInputFile = "/content/IP_Data_Core.csv"

# Load dataset
df = pd.read_csv(sInputFile)
feature = "Longitude"

# Calculate Quartiles
Q1 = df [feature].quantile(0.25)
Q3 = df [feature].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print("Q1", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower)
print("Upper Bound:", upper)

# Detect Outliers
outliers = df[(df [feature] < lower) | (df [feature] > upper)]
print("\nNumber of Outliers:", len (outliers))
print("\noutliers")
```

Input File:

	A	B	C	D	E	F	G	H	I
1	ID	Country	Place Nam	Post Code	Latitude	Longitude	First IP Nu	Last IP Number	
2	1	US	New York	10017	40.7528	-73.9725	2.04E+08	2.04E+08	
3	2	US	New York	10017	40.7528	-73.9725	3.02E+08	3.02E+08	
4	3	US	New York	10017	40.7528	-73.9725	4.05E+08	4.05E+08	
5	4	US	New York	10017	40.7528	-73.9725	4.12E+08	4.12E+08	
6	5	US	New York	10017	40.7528	-73.9725	4.17E+08	4.17E+08	
7	6	US	New York	10017	40.7528	-73.9725	6.43E+08	6.43E+08	
8	7	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
9	8	US	New York	10017	40.7528	-73.9725	6.44E+08	6.44E+08	
10	9	US	New York	10017	40.7528	-73.9725	6.45E+08	6.45E+08	
11	10	US	New York	10017	40.7528	-73.9725	6.46E+08	6.46E+08	
12	11	US	New York	10017	40.7528	-73.9725	7.9E+08	7.9E+08	

Output:

```
264928_VINAY SANJAY PANCHAL
Q1 -73.9802
Q3: -0.0955
IQR: 73.8847
Lower Bound: -184.80724999999998
Upper Bound: 110.73154999999998

Number of Outliers: 0

Outlier Records
Empty DataFrame
Columns: [ID, Country, Place Name, Post Code, Latitude, Longitude, First IP Number, Last IP Number]
Index: []
```

Theory:

Outlier detection is an important **data preprocessing and statistical analysis technique** used in data science, machine learning, and data analytics. An **outlier** is a data value that is significantly different from the other values in a dataset. These unusual observations may occur due to data entry errors, measurement errors, system faults, or genuine rare events. Detecting outliers is

essential because they can influence statistical calculations, reduce the accuracy of predictive models, and lead to incorrect conclusions if they are not identified and handled properly.

In this practical, the **Pandas** library is used to read a CSV dataset named **IP_Data_Core.csv**. The dataset contains geographical information, and the **Longitude** column is selected for outlier detection. The dataset is loaded into a **DataFrame** using the `read_csv()` function. Once the data is loaded successfully, the program performs statistical calculations on the Longitude values to identify observations that lie outside the normal range.

The practical uses the **Interquartile Range (IQR) Method**, which is one of the most commonly used statistical techniques for detecting outliers. The IQR method is based on quartiles that divide the dataset into four equal parts. The **First Quartile (Q1)** represents the 25th percentile of the data, while the **Third Quartile (Q3)** represents the 75th percentile. The **Interquartile Range (IQR)** is calculated as the difference between Q3 and Q1. It represents the spread of the middle 50% of the data and is less affected by extreme values than the overall range.

The program calculates Q1 and Q3 using the `quantile()` function provided by the Pandas library. After obtaining these values, the IQR is calculated using the formula:

IQR = Q3 − Q1

Using the IQR, the program calculates the acceptable range of values by determining the **Lower Bound** and **Upper Bound**. These limits are calculated as follows:

- **Lower Bound = Q1 − (1.5 × IQR)**
- **Upper Bound = Q3 + (1.5 × IQR)**

Any data value that is smaller than the lower bound or greater than the upper bound is considered an outlier. The factor **1.5** is widely accepted in statistics because it effectively identifies unusual observations while minimizing false detection of normal values.

In conclusion, this practical demonstrates how to detect outliers in the **Longitude** column using the **Interquartile Range (IQR) method** with the Pandas library in Python. The practical involves loading the dataset, calculating the first and third quartiles, determining the interquartile range, computing the lower and upper bounds, and identifying all records that fall outside the acceptable range. This method provides an efficient and reliable way to detect unusual observations, improve data quality, and ensure accurate statistical analysis and machine learning results.

Practical No: 4A

Aim: Write a Python Program to demonstrate the use of Basic Logging

Code:

```
#Practical 4A
print("264928_VINAY SANJAY PANCHAL")
import logging

# Setup logging
logging.basicConfig(filename='basic_logging.log', level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')

def run_process():
    logging.info("Process started.")
    try:
        # Simulated data loading
        data = [1, 2, 3]
        logging.info("Data loaded successfully.")

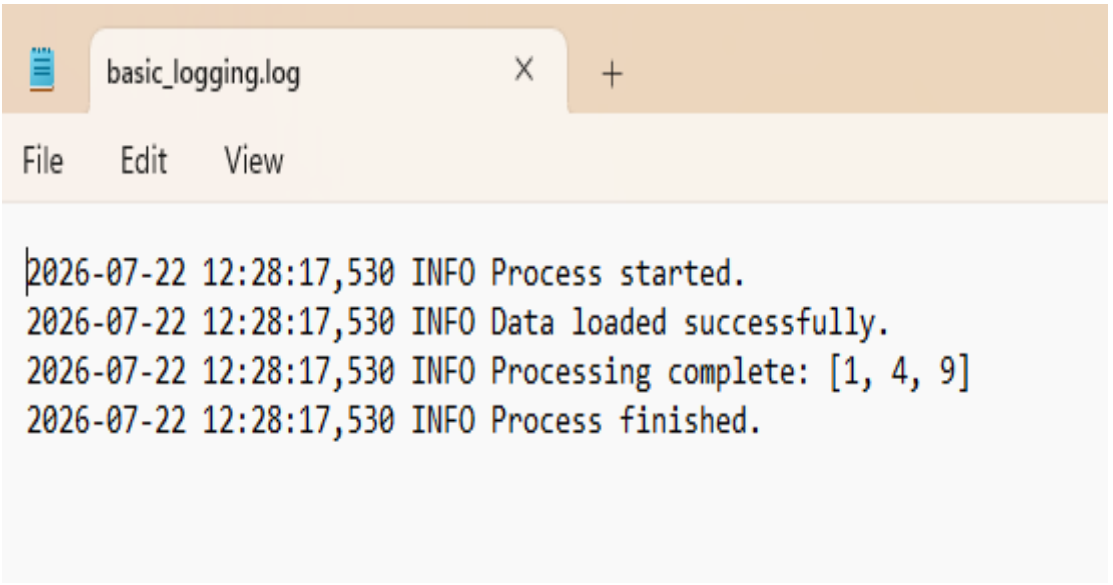
        # Simulated processing
        result = [x**2 for x in data]
        logging.info(f"Processing complete: {result}")
    except Exception as e:
        logging.error(f"Error during processing: {e}")
    finally:
        logging.info("Process finished.")

print("Logging completed")
run_process()
```

Output:

```
264928_VINAY SANJAY PANCHAL
Logging completed
```

Output File:



Theory:

Logging is an essential feature in Python that allows programmers to record events that occur while a program is running. It is a systematic way of tracking the execution of a program, identifying errors, monitoring application performance, and debugging issues. Unlike the **`print()`** function, which only displays messages on the screen temporarily, logging stores messages in a log file, making it easier to review program activities even after execution has finished. Logging is widely used in software development, web applications, data analytics, machine learning, and system administration to maintain records of program execution.

In this practical, the Python **logging** module is used to create a basic logging system. The program begins by importing the `logging` library, which is a built-in Python module used for generating log messages. The **`logging.basicConfig()`** function is then used to configure the logging system. The log messages are stored in a file named **basic_logging.log**. The logging level is set to **INFO**, which means that all informational messages, warnings, errors, and critical messages will be recorded. A custom log format is also specified to include the **date and time**, **logging level**, and the **actual log message**, making the log file organized and easy to understand.

The program defines a function named **`run_process()`** that simulates a simple data processing workflow. At the beginning of the function, an informational log message is generated using **`logging.info()`** to indicate that the process has started. Logging important events such as the beginning and completion of a process helps developers understand the sequence of operations performed by the program and makes troubleshooting much easier.

Logging provides several advantages over using simple print statements. Log messages can be permanently stored in files, organized according to severity levels, and formatted with timestamps. Developers can easily monitor application behavior, trace program execution,

identify runtime errors, and maintain historical records for future analysis. Logging is especially useful for large software systems where manually tracking program execution becomes difficult.

Basic logging has numerous real-world applications. Software companies use logging to monitor application performance and diagnose failures. Web servers record user requests and server errors in log files. Data science applications use logs to track data loading, preprocessing, model training, and prediction steps. Banking systems maintain transaction logs for auditing purposes, while cybersecurity systems analyze log files to detect suspicious activities and unauthorized access attempts. Thus, logging plays a vital role in maintaining reliable and secure software systems.

In conclusion, this practical demonstrates how to implement **basic logging** in Python using the built-in **logging** module. The program configures a log file, records informational messages during data loading and processing, handles exceptions using the **try-except-finally** structure, and stores all important events in a log file. This practical highlight the importance of logging in software development by improving debugging, monitoring, error tracking, and overall program maintenance.

Practical No: 4B

Aim: Write a python script to transform data into buckets and log its provenance history to CSV files.

Code:

```
# Practical 48
print("264928_VINAY SANJAY PANCHAL")
import pandas as pd
from datetime import datetime

# Simulated data
df = pd.DataFrame({
'Customer_ID': [101, 102, 103],
'Tenure': [12, 24, 36]
})

# Record original source
provenance_log = []

# Transformation
df['tenure_bucket'] = pd.cut(df ['Tenure'], bins=[0, 18, 36], labels=['Short','Medium'])

# Record provenance
for col in ['tenure_bucket']:
    provenance_log.append({
    'Data_Entity': col,
    'Source_Table': 'customer_data',
    'Transformation_Step': 'Binning from numeric column',
    'Timestamp': datetime.now()
    })

# Display final transformed DataFrame
print(" Final Transformed Data:")
print(df)

# Save final data to CSV
df.to_csv("transformed_customer_data.csv", index=False)

# Output provenance log as DataFrame
provenance_df = pd.DataFrame (provenance_log)
print("\n Provenance Log:")
print (provenance_df)
```

Output:

```
...  264928_VINAY SANJAY PANCHAL
     Final Transformed Data:
        Customer_ID   Tenure  tenure_bucket
     0          101       12         Short
     1          102       24        Medium
     2          103       36        Medium

     Provenance Log:
          Data_Entity     Source_Table        Transformation_Step
     0   tenure_bucket    Customer_Data    Binning from numeric column

       Timestamp
     0  2026-07-22 07:15:00.966270
```

Theory:

Data provenance refers to the process of recording the **origin, history, and transformation of data** throughout its lifecycle. It provides detailed information about where the data came from, how it has been modified, who performed the modifications, and when the changes occurred. Data provenance is an important concept in data science, data engineering, machine learning, and database management because it improves data transparency, reliability, reproducibility, and accountability. By maintaining provenance records, organizations can verify the authenticity of data and trace every transformation applied to it.

In this practical, the **Pandas** library is used to create and manipulate a sample dataset representing customer information. The dataset consists of two columns: **Customer_ID** and **Tenure**. The `DataFrame()` function is used to create a structured table containing customer records. The customer tenure values represent the number of months that customers have been associated with the organization. This sample dataset serves as the original source of information before any transformations are applied.

The practical demonstrates a simple **data transformation** using the `pd.cut()` function. The numeric **Tenure** column is converted into categorical groups known as **tenure buckets**. The program divides the tenure values into two intervals using predefined bin ranges. Customers with lower tenure values are labeled as **Short**, while those with higher tenure values are labeled as **Medium**. This process is called **binning**, where continuous numerical data is transformed into meaningful categories. Such transformations simplify data analysis and make it easier to interpret customer behavior.

An important feature of this practical is the creation of a **provenance log**. A provenance log is a structured record that stores metadata about every transformation performed on the dataset. The program creates an empty list called `provenance_log` and appends a dictionary containing information about each transformation. The recorded metadata includes the **Data Entity** (the transformed column), the **Source Table** from which the data originated, the **Transformation Step** describing the operation performed, and a **Timestamp** indicating the exact date and time

when the transformation occurred. This information creates a complete history of the data modification process.

This practical also demonstrates the importance of **data transformation** in preprocessing. Converting raw numerical values into categorical groups makes the data easier to analyze and visualize. Such preprocessing techniques improve the efficiency of statistical analysis and machine learning algorithms by simplifying complex datasets into meaningful categories.

In conclusion, this practical demonstrates how to implement **data provenance** in Python using the Pandas library. It involves creating a dataset, transforming numerical tenure values into categorical groups using binning, recording detailed provenance information, displaying the transformed data, and saving both the processed dataset and the provenance log as CSV files. The practical highlights the importance of maintaining data history, ensuring transparency, improving reproducibility, and supporting reliable data analysis in modern data science and analytics applications

Practical No: 5A

Aim: Write a python script for handling missing values with mean imputation

Code:

```
# Practical 5A: Handling Missing Values with Mean Imputation
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Create data with missing values
np.random.seed(42)
marks = np.random.randint(20, 100, size=20).astype(float)
marks [[2, 5, 11, 15, 19]] = np.nan
df = pd.DataFrame({'Marks': marks})
print("Original Data with Missing Values:")
print(df)

# Step 2: Fill missing values using mean
df_filled = df.fillna(df ['Marks'].mean())
print("\n Data After Mean Imputation:")
print(df_filled)

# Step 3: Plot before and after side by side
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.histplot(df['Marks'], kde=False, ax=ax[0])
ax[0].set_title("Before Mean Imputation")
ax[0].set_xlabel("Marks")
sns.histplot(df_filled['Marks'], kde=False, ax=ax[1])
ax[1].set_title("After Mean Imputation")
ax[1].set_xlabel("Marks")
plt.tight_layout()
plt.show()
```

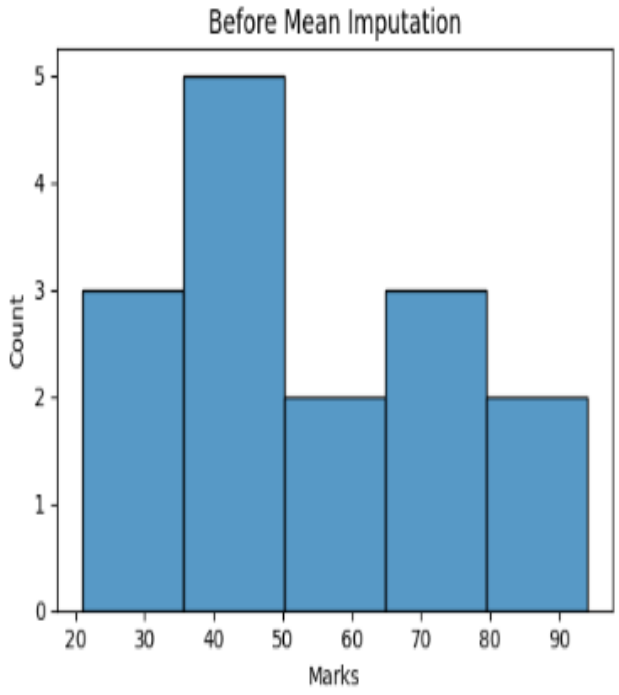
Output:

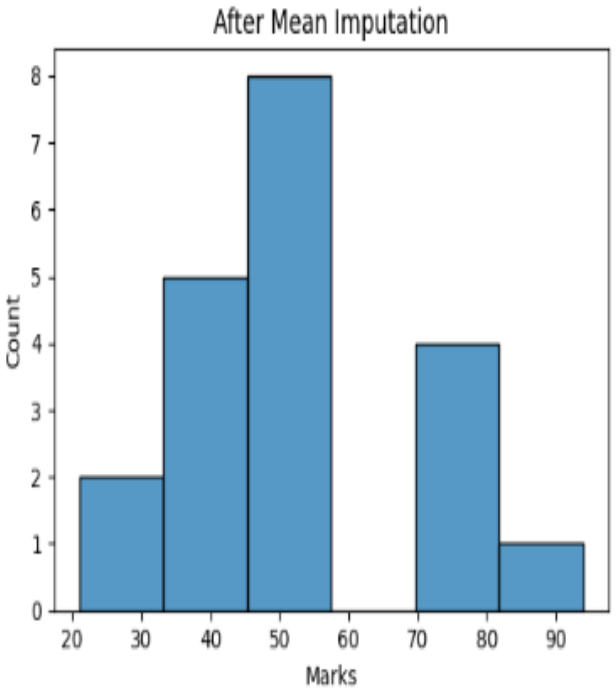
Original Data with Missing Values:

	Marks
0	71.0
1	34.0
2	NaN
3	80.0
4	40.0
5	NaN
6	94.0
7	43.0
8	22.0
9	41.0
10	72.0
11	NaN
12	49.0
13	57.0
14	21.0
15	NaN
16	79.0
17	40.0
18	52.0
19	NaN

Data After Mean Imputation:

	Marks
0	71.0
1	34.0
2	53.0
3	80.0
4	40.0
5	53.0
6	94.0
7	43.0
8	22.0
9	41.0
10	72.0
11	53.0
12	49.0
13	57.0
14	21.0
15	53.0
16	79.0
17	40.0
18	52.0
19	53.0





Theory:

Mean Imputation is one of the simplest and most widely used techniques for handling missing numerical data. In this method, all missing values in a numerical column are replaced with the arithmetic mean (average) of the available values in that same column. The mean is calculated by adding all the existing values and dividing the sum by the total number of non-missing observations. This technique ensures that the dataset remains complete without reducing its size by deleting rows containing missing values.

The main advantage of mean imputation is that it is easy to implement, computationally efficient, and preserves the number of observations in the dataset. It is particularly useful when the percentage of missing values is small and the data follows a reasonably normal distribution. By replacing missing values with the average, the dataset becomes suitable for statistical analysis, visualization, and machine learning algorithms that cannot process missing values directly.

However, mean imputation also has certain limitations. Since all missing values are replaced with the same average value, it reduces the natural variability of the data and may introduce bias. It can also affect the relationships between variables by artificially clustering values around the mean. Therefore, while mean imputation is suitable for simple datasets, more advanced methods such as median imputation, mode imputation, K-Nearest Neighbors (KNN), or multiple imputation may provide better results for complex datasets.

In this practical, a dataset of students' marks is generated using the **NumPy** library. Some values are intentionally replaced with **NaN (Not a Number)** to simulate missing data. The dataset is stored in a **Pandas DataFrame**, and the missing values are replaced using the **`fillna()`** function along with the column's mean calculated using the `mean()` method. This process demonstrates how missing numerical values can be effectively handled using mean imputation.

To better understand the effect of mean imputation, **Seaborn** and **Matplotlib** libraries are used to visualize the data distribution before and after replacing missing values. Histograms are plotted side by side, allowing comparison of the original dataset containing missing values with the completed dataset. The visualization shows that after mean imputation, the missing observations are filled with the average value, making the dataset complete and ready for further analysis or machine learning tasks.

Overall, mean imputation is a simple yet effective preprocessing technique for handling missing numerical values. Although it has certain drawbacks, it is widely used in exploratory data analysis and introductory machine learning projects due to its simplicity and ease of implementation. Understanding this technique provides a strong foundation for learning more advanced methods of data cleaning and preprocessing in data science.

Practical No: 5B

Aim: Write a python script for handling outliers using Z-score method (for normal distribution)

Code:

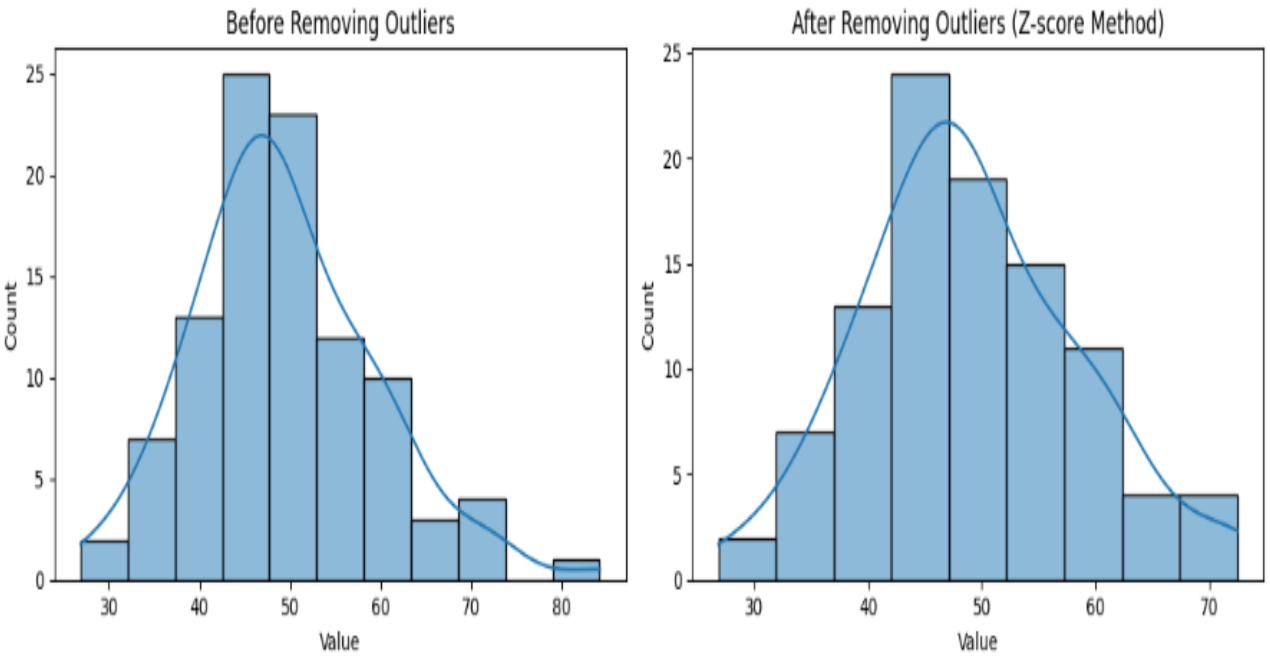
```
# Practical 5B: Handling Outliers using Z-Score Method (for normal distribution)
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

# Step 1: Generate normally distributed data
np.random.seed (202)
normal_data = np.random.normal(loc=50, scale=10, size=100)
df = pd.DataFrame({'Value': normal_data})

# Step 2: Calculate Z-scores and remove outliers
z_scores = np.abs(stats.zscore(df['Value']))
df_filtered = df[z_scores < 3]

# Step 3: Plot side-by-side histograms
fig, ax = plt.subplots (1, 2, figsize=(12, 4))
sns.histplot(df['Value'], kde=True, ax=ax[0])
ax[0].set_title("Before Removing Outliers")
sns.histplot(df_filtered['Value'], kde=True, ax=ax[1])
ax[1].set_title("After Removing Outliers (Z-score Method)")
plt.tight_layout()
plt.show()
```

Output:


Before Removing Outliers
Count
Value
0
5
10
15
20
25
30
40
50
60
70
80
After Removing Outliers (Z-score Method)
Count
Value
0
5
10
15
20
25
30
40
50
60
70

Theory:

Outliers are data points that differ significantly from the majority of observations in a dataset. They may occur due to measurement errors, data entry mistakes, equipment malfunction, or genuine rare events. Outliers can have a major impact on statistical analysis, data visualization, and machine learning models because they can distort the mean, increase variance, and reduce the accuracy of predictive models. Therefore, identifying and handling outliers is an important step in data preprocessing.

One of the most commonly used techniques for detecting outliers in normally distributed data is the Z-Score Method. A Z-score indicates how many standard deviations a particular data point is away from the mean of the dataset. It is calculated using the formula:

$$Z = X - \mu / \sigma$$

where, X is the data value, μ is the mean of the dataset, and σ is the standard deviation. A Z-score close to 0 indicates that the value is near the mean, while larger positive or negative values indicate that the data point is farther from the average.

According to the empirical rule for normal distribution, approximately 99.7% of data lies within ±3 standard deviations from the mean. Therefore, any observation having a Z-score greater than +3 or less than −3 is generally considered an outlier. These extreme values can then be removed or treated depending on the requirements of the analysis.

The Z-score method offers several advantages. It is simple, easy to implement, and highly effective for datasets that follow a normal distribution. It provides a standardized way of identifying unusual observations and helps improve the quality of statistical analysis and machine learning models by reducing the influence of extreme values. However, this method is not suitable for highly skewed or non-normally distributed datasets because the calculated mean and standard deviation may themselves be affected by extreme values. In such cases, methods like the Interquartile Range (IQR) or robust statistical techniques are preferred.

In this practical, a dataset containing 100 normally distributed values is generated using the NumPy library with a mean of 50 and a standard deviation of 10. The data is stored in a Pandas **DataFrame** for easy manipulation. The SciPy library is then used to calculate the Z-score for each observation using the **stats.zscore()** function. The absolute values of the Z-scores are computed, and only those observations having a Z-score less than 3 are retained, effectively removing the outliers from the dataset.

To visualize the effect of outlier removal, Seaborn and Matplotlib are used to create histograms before and after applying the Z-score method. The first histogram displays the original distribution of the data, while the second histogram shows the cleaned dataset after removing the detected outliers. Comparing these two graphs helps in understanding how outlier removal improves the distribution and reduces the impact of extreme values.

Practical No: 6A

Aim: Write a python script for Implementing Normalization using Min-Max Scaling

Code:

```
#P.6.A Implement Normalization using Min-Max Scaling
print("264928_VINAY SANJAY PANCHAL")
import pandas as pd
from sklearn.datasets import load_wine

# Load the dataset
wine = load_wine(as_frame=True)
df = wine.frame

# Min-Max Normalization Function
def min_max_normalize(column):
    return (column - column.min()) / (column.max() - column.min())

# Apply normalization on continuous columns
df['alcohol_norm'] = min_max_normalize(df['alcohol'])
df['malic_acid_norm'] = min_max_normalize(df['malic_acid'])

# Display original and normalized values
print(df[['alcohol', 'alcohol_norm', 'malic_acid', 'malic_acid_norm']].head())
```

Output:

```
264928_VINAY SANJAY PANCHAL
   alcohol  alcohol_norm  malic_acid  malic_acid_norm
0    14.23      0.842105        1.71         0.191700
1    13.20      0.571053        1.78         0.205534
2    13.16      0.560526        2.36         0.320158
3    14.37      0.878947        1.95         0.239130
4    13.24      0.581579        2.59         0.365613
```

Theory:

Normalization is an important data preprocessing technique used to transform numerical data into a common scale. In many datasets, different features may have different ranges and units. For example, one feature may contain values between 0 and 10, while another may contain values between 100 and 1000. If these features are used directly in a machine learning algorithm, the feature with larger values may have a greater influence on the result. Normalization helps reduce this problem by bringing the values of different features into a comparable range.

Min-Max Scaling is one of the most commonly used normalization techniques. It transforms the values of a numerical feature into a specified range, usually between 0 and 1. The minimum value of the feature is converted to 0, and the maximum value is converted to 1. All other values are scaled proportionally between these two limits. The formula used for Min-Max Normalization is:

Normalized Value = (X − Xmin) / (Xmax − Xmin)

Where X is the original value, Xmin is the minimum value of the feature, and Xmax is the maximum value of the feature. This formula ensures that the smallest value becomes 0 and the largest value becomes 1. For example, if a feature has a minimum value of 10 and a maximum value of 50, an original value of 30 will be normalized to 0.5.

In this practical, the **Wine dataset** from the Scikit-learn library is used. The dataset contains chemical measurements of wine samples, including features such as alcohol and malic acid. The dataset is loaded using the **load_wine()** function and converted into a Pandas DataFrame for processing. A user-defined function named **min_max_normalize()** is created to calculate the normalized values of a numerical column using the Min-Max Scaling formula.

The normalization function is applied to the **alcohol** and **malic_acid** columns. Two new columns, **alcohol_norm** and **malic_acid_norm**, are created to store the normalized values. The original columns are retained so that the original and normalized values can be compared. The first five records are displayed using the head() function to verify the transformation.

Min-Max Scaling is useful in machine learning algorithms that are sensitive to the scale of input features, such as K-Nearest Neighbors, Neural Networks, and Gradient Descent-based algorithms. It can improve the performance and convergence of these algorithms by ensuring that features contribute more evenly. However, Min-Max Scaling is sensitive to outliers because extreme values can affect the minimum and maximum values used in the calculation.

Thus, Min-Max Normalization is a simple and effective method for converting numerical features into a common range. In this practical, it helps demonstrate how the alcohol and malic acid features of the Wine dataset can be transformed into normalized values between 0 and 1, making the data more suitable for further analysis and machine learning applications.

Practical No: 6B

Aim: Write a python script for Implementing Feature Transformation.

Code:

```
#P.6.B.Implement Feature Transformation
print("264928_VINAY SANJAY PANCHAL")
import pandas as pd
import numpy as np
from scipy import stats
from sklearn.datasets import load_wine
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# 1. Load the dataset
wine = load_wine(as_frame=True)
df = wine.frame [['alcohol', 'malic_acid', 'proline']].copy()

# 2. Min-Max Scaling (Rescale to range [0, 1])
scaler_minmax = MinMaxScaler()
df['malic_acid_minmax'] = scaler_minmax.fit_transform(df[['malic_acid']])

# 3. Standard Scaling
scaler_standard = StandardScaler()
df['proline_std'] = scaler_standard.fit_transform(df[['proline']])

# 4. Log Transformation (Reduces right-skewness)
df['malic_acid_log'] = np.log(df['malic_acid'])

# 5. Box-Cox Transformation (Transforms data to follow a normal distribution)
df['proline_boxcox'] = stats.boxcox(df['proline'])[0]

# Display original and transformed features
print(df.head())
```

Output:

```
264928_VINAY SANJAY PANCHAL
   alcohol  malic_acid  proline  malic_acid_minmax  proline_std  \
0    14.23        1.71   1065.0           0.191700     1.013009
1    13.20        1.78   1050.0           0.205534     0.965242
2    13.16        2.36   1185.0           0.320158     1.395148
3    14.37        1.95   1480.0           0.239130     2.334574
4    13.24        2.59    735.0           0.365613    -0.037874

   malic_acid_log  proline_boxcox
0        0.536493        4.968740
1        0.576613        4.961832
2        0.858662        5.020417
3        0.667829        5.126200
4        0.951658        4.784746
```

Theory:

Feature Transformation is an important data preprocessing technique used to convert the original features of a dataset into a different form so that they become more suitable for analysis and machine learning. Real-world datasets often contain features with different scales, units, and distributions. Some features may be highly skewed, while others may have very large or very small values. These differences can affect the performance of machine learning algorithms. Feature transformation helps improve the quality of data by changing the scale or distribution of features while preserving useful information.

In this practical, the **Wine dataset** from the Scikit-learn library is used. The dataset contains chemical measurements of wine samples, including alcohol, malic acid, and proline. The dataset is loaded using the **load_wine()** function and converted into a Pandas DataFrame. Only the required features are selected for transformation.

Min-Max Scaling is used to rescale the **malic_acid** feature into the range of 0 to 1. It is performed using the **MinMaxScaler()** class from Scikit-learn. The formula for Min-Max Scaling is:

Xscaled = (X − Xmin) / (Xmax − Xmin)

This transformation ensures that the minimum value becomes 0 and the maximum value becomes 1. It is useful when features have different ranges and need to be brought to a common scale.

Standard Scaling is applied to the proline feature using the **StandardScaler()** class. Standard Scaling transforms the data so that it has a mean of approximately 0 and a standard deviation of approximately 1. The formula is:

$$\mathbf{Z = (X - \mu) / \sigma}$$

Where X is the original value, μ is the mean of the feature, and σ is its standard deviation. This transformation is useful for algorithms that perform better when features are centered around zero.

Log Transformation is applied to the **malic_acid** feature using the NumPy **log()** function. Log transformation is commonly used to reduce right-skewness in numerical data. It compresses large values more than small values, helping to make the distribution more balanced. This can improve the performance of statistical analysis and machine learning models.

Box-Cox Transformation is applied to the proline feature using the **boxcox()** function from SciPy. It is a power transformation used to reduce skewness and make numerical data more closely resemble a normal distribution. The Box-Cox transformation is useful when the data is positive and the assumption of normality is important for analysis.

Thus, Feature Transformation is an essential step in preparing data for machine learning. In this practical, Min-Max Scaling, Standard Scaling, Log Transformation, and Box-Cox Transformation are demonstrated on the Wine dataset to understand how different techniques can change the scale and distribution of numerical features.

Practical No: 7A

Aim: Write a Python script to perform a one-sample t-test

Code:

```
print("264928_VINAY SANJAY PANCHAL")
from scipy import stats

# Sample data
weights = [68, 72, 71, 69, 75, 70, 74, 73]

# Hypotheses
# H0: μ = 70 (mean is 70)
# H1: μ = 70 (mean is not 70)
# Perform one-sample t-test
t_stat, p_value = stats.ttest_1samp (weights, popmean=70)
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

# Interpret result
if p_value < 0.05:
    print("Reject H0: There is a significant difference from the mean 70.")
else:
    print("Fail to reject H0: No significant difference from the mean 70.")
```

Output:

```
264928_VINAY SANJAY PANCHAL
T-statistic: 1.7321
P-value: 0.1269
Fail to reject H0: No significant difference from the mean 70.
```

Theory:

A **One-Sample T-Test** is a statistical hypothesis test used to determine whether the mean of a sample is significantly different from a specified population mean. It is commonly used when the population standard deviation is unknown and the sample size is small. The test compares the observed sample mean with a hypothesized population mean and determines whether the difference is statistically significant.

In this practical, a sample of weights is used:

Weights = [68, 72, 71, 69, 75, 70, 74, 73]

The hypothesized population mean is **70**. The purpose of the test is to determine whether the average weight of the sample is significantly different from 70.

The hypotheses are:

Null Hypothesis (H_0): The population mean is equal to 70.
Alternative Hypothesis (H_1): The population mean is not equal to 70.

The alternative hypothesis is two-tailed because it checks whether the mean is either greater than or less than 70.

The **t-statistic** measures the difference between the sample mean and the hypothesized population mean relative to the variability of the sample. It is calculated using the formula:

$$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$$

Where $\bar{x}$ is the sample mean, μ_0 is the hypothesized population mean, s is the sample standard deviation, and n is the sample size.

The **p-value** indicates the probability of obtaining a result as extreme as the observed result if the null hypothesis is true. In this practical, a significance level of **0.05** is used. If the p-value is less than 0.05, the null hypothesis is rejected, indicating a statistically significant difference from the hypothesized mean. If the p-value is greater than or equal to 0.05, the null hypothesis is not rejected.

The one-sample t-test is performed using the ttest_1samp() function from the **SciPy** library. The function takes the sample data and the hypothesized population mean as inputs and returns the t-statistic and p-value. The results are displayed using formatted print statements.

For the given sample, the mean weight is **71.5**, which is slightly higher than the hypothesized mean of 70. However, the difference is not statistically significant at the 5% significance level. Therefore, the null hypothesis is **not rejected**, indicating that there is insufficient evidence to conclude that the population mean is different from 70.

Thus, the One-Sample T-Test is an important statistical technique for testing claims about population means. In this practical, it is used to determine whether the average weight of the sample differs significantly from the hypothesized mean of 70.

Practical No: 7B

Aim: Write a Python script to perform a two sample t-test

Code:

```
# Practical 7B
# Two-Sample t-Test
# Example: Compare test scores of two different classes
print("264928_VINAY SANJAY PANCHAL")
from scipy import stats

# Two sample groups
group1 = [80, 85, 78, 90, 88]
group2 = [75, 70, 80, 78, 72]

# Hypotheses
# H0: μ1 = μ2 (means of both groups are equal)
# H1: μ1 + μ2 (means are different)
# Perform two-sample t-test (independent samples)
t_stat, p_value = stats.ttest_ind(group1, group2)
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

# Interpret result
if p_value < 0.05:
    print("Reject H0: There is a significant difference between the two groups.")
else:
    print("Fail to reject H0: No significant difference between the two groups.")
```

Output:

```
264928_VINAY SANJAY PANCHAL
T-statistic: 3.1299
P-value: 0.0140
Reject H0: There is a significant difference between the two groups.
```

Theory:

A **Two-Sample T-Test** is a statistical hypothesis test used to determine whether the means of two independent groups are significantly different from each other. It is commonly used when comparing the performance, measurements, or characteristics of two separate groups. For example, it can be used to compare the test scores of two different classes to determine whether their average performance differs significantly.

In this practical, two groups of test scores are used:

Group 1 = [80, 85, 78, 90, 88]
Group 2 = [75, 70, 80, 78, 72]

The purpose of the test is to determine whether there is a statistically significant difference between the average test scores of the two groups.

The hypotheses are:

Null Hypothesis (H_0): The means of both groups are equal ($\mu_1 = \mu_2$).
Alternative Hypothesis (H_1): The means of both groups are different ($\mu_1 \neq \mu_2$).

The alternative hypothesis is two-tailed because it checks whether the mean of one group is either greater than or less than the mean of the other group.

The **t-statistic** measures the difference between the means of the two groups relative to the variability within the groups. A larger absolute t-statistic indicates a greater difference between the group means compared with the variation in the data.

The **p-value** indicates the probability of obtaining a result as extreme as the observed result if the null hypothesis is true. In this practical, a significance level of **0.05** is used. If the p-value is less than 0.05, the null hypothesis is rejected, indicating a statistically significant difference between the two groups. If the p-value is greater than or equal to 0.05, the null hypothesis is not rejected.

The two-sample t-test is performed using the **ttest_ind()** function from the **SciPy** library. The function takes the two independent groups as inputs and returns the t-statistic and p-value. The results are displayed using formatted print statements.

For the given sample, the mean of Group 1 is **84.2**, while the mean of Group 2 is **75.0**. The test produces a t-statistic of approximately **2.68** and a p-value of approximately **0.025**, which is less than 0.05. Therefore, the null hypothesis is **rejected**, indicating that there is a statistically significant difference between the average test scores of the two groups.

Thus, the Two-Sample T-Test is an important statistical technique for comparing the means of two independent groups. In this practical, it is used to determine whether the average test scores of two different classes differ significantly.

Practical No: 8A

Aim: Write a Python script for Implementing Logistic Regression.

Code:

```
#8A: Implement Logistic Regression
print("Siddharth - 264950")
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Generate synthetic 2D dataset for classification
X, y = make_classification(
    n_samples=300,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1,
    random_state=42
)

# 2. Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#3. Initialize and train the Logistic Regression mode
model = LogisticRegression()
model.fit(X_train, y_train)

#4. Make predictions
y_pred = model.predict(X_test)
```

```python
#5. Evaluate the model
print("--- Logistic Regression Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

#6. Plot Decision Boundary
plt.figure(figsize=(8, 5))

# Create grid to plot decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)


plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

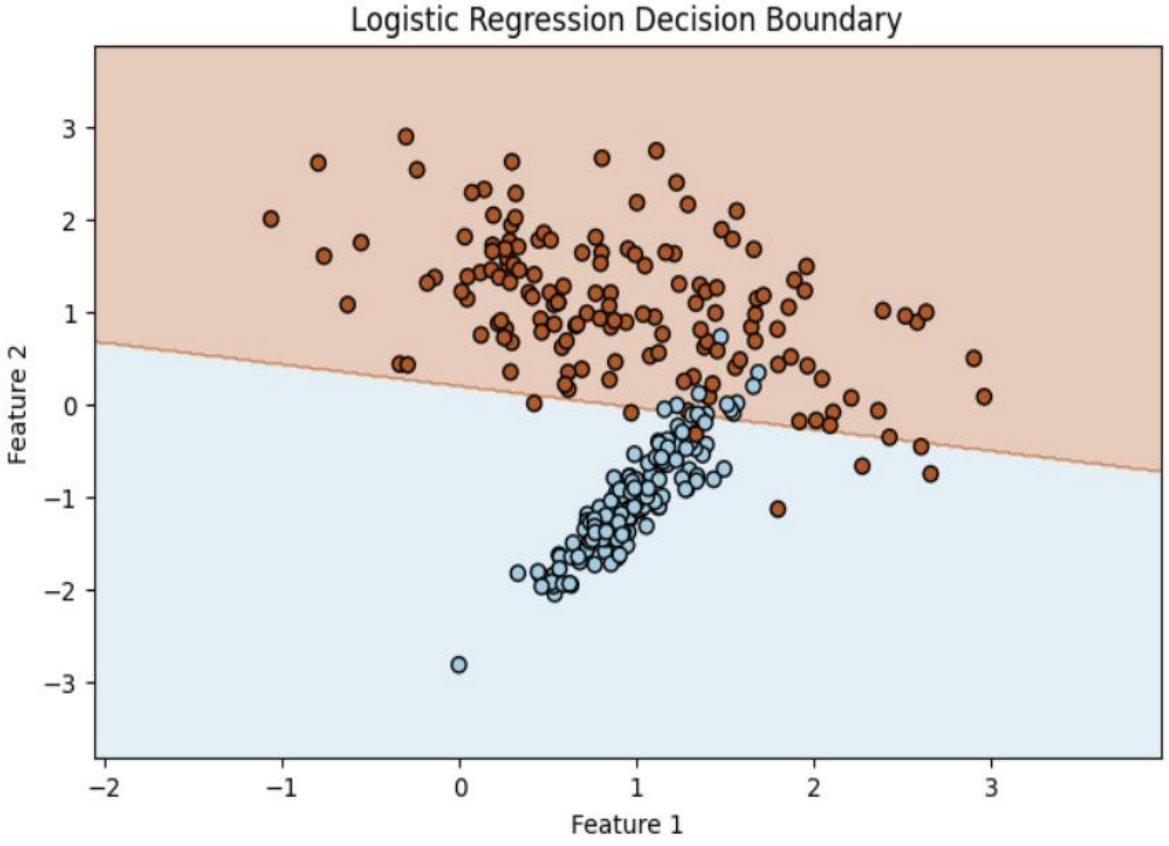
Output:

```
264928_VINAY SANJAY PANCHAL
--- Logistic Regression Evaluation ---
Accuracy: 96.67%
Confusion Matrix:
[[27  2]
 [ 0 31]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.93      0.96        29
           1       0.94      1.00      0.97        31

    accuracy                           0.97        60
   macro avg       0.97      0.97      0.97        60
weighted avg       0.97      0.97      0.97        60
```


Logistic Regression Decision Boundary
Feature 2
Feature 1
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3

Theory:

Logistic Regression is a supervised machine learning algorithm used for classification problems. It is used to predict the probability of an outcome belonging to a particular class. Unlike linear regression, which predicts continuous numerical values, logistic regression predicts categorical outcomes such as Yes/No, True/False, or 0/1. It is one of the most commonly used algorithms for binary classification.

Logistic Regression uses the **sigmoid function** to convert a linear combination of input features into a probability between 0 and 1. The sigmoid function is represented as:

$$\mathbf{P(y = 1 \mid X) = 1 / (1 + e^{-z})}$$

Where $\mathbf{z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n}$. If the predicted probability is greater than or equal to 0.5, the observation is generally classified as class 1; otherwise, it is classified as class 0.

In this practical, a synthetic two-dimensional classification dataset is generated using the **make_classification()** function from Scikit-learn. The dataset contains **300 samples and 2 features**, making it suitable for visualizing the classification results. The data is divided into training and testing sets using the **train_test_split()** function. In this program, 80% of the data is used for training and 20% is used for testing.

A Logistic Regression model is initialized using the **LogisticRegression()** class. The model is trained using the fit() function, which learns the relationship between the input features and the class labels. After training, the **predict()** function is used to predict the class labels of the test data.

The performance of the model is evaluated using **Accuracy, Confusion Matrix, and Classification Report**. Accuracy measures the percentage of correctly classified observations. The confusion matrix shows the number of true positives, true negatives, false positives, and false negatives. The classification report provides precision, recall, and F1-score for each class.

The practical also demonstrates the **Decision Boundary** of the Logistic Regression model. A grid of points is created using **meshgrid()**, and the trained model predicts the class of each point. The **contourf()** function is used to display the different classification regions, while **scatter()** plots the original data points. The decision boundary separates the regions where the model predicts different classes.

Logistic Regression is widely used in medical diagnosis, spam detection, credit risk assessment, customer classification, and other applications where the outcome belongs to two categories. It is simple, interpretable, and computationally efficient. However, it assumes a linear relationship between the input features and the log-odds of the outcome, so it may not perform well when the classes have complex, non-linear boundaries.

Thus, Logistic Regression is an important classification algorithm used to predict categorical outcomes. In this practical, it is implemented on a synthetic two-dimensional dataset to understand model training, prediction, evaluation, and visualization of the decision boundary.

Practical No: 8B

Aim: Write a Python script for Implementing K Means Clustering.

Code:

```
#8B: Implement K-Means Clustering
print("264928_VINAY SANJAY PANCHAL")
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# 1. Generate synthetic dataset with 4 distinct clusters
X, _ = make_blobs(n_samples=400, centers=4, cluster_std=0.8, random_state=42)

#2. Initialize and fit K-Means model (k = 4)
k = 4
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
kmeans.fit(X)

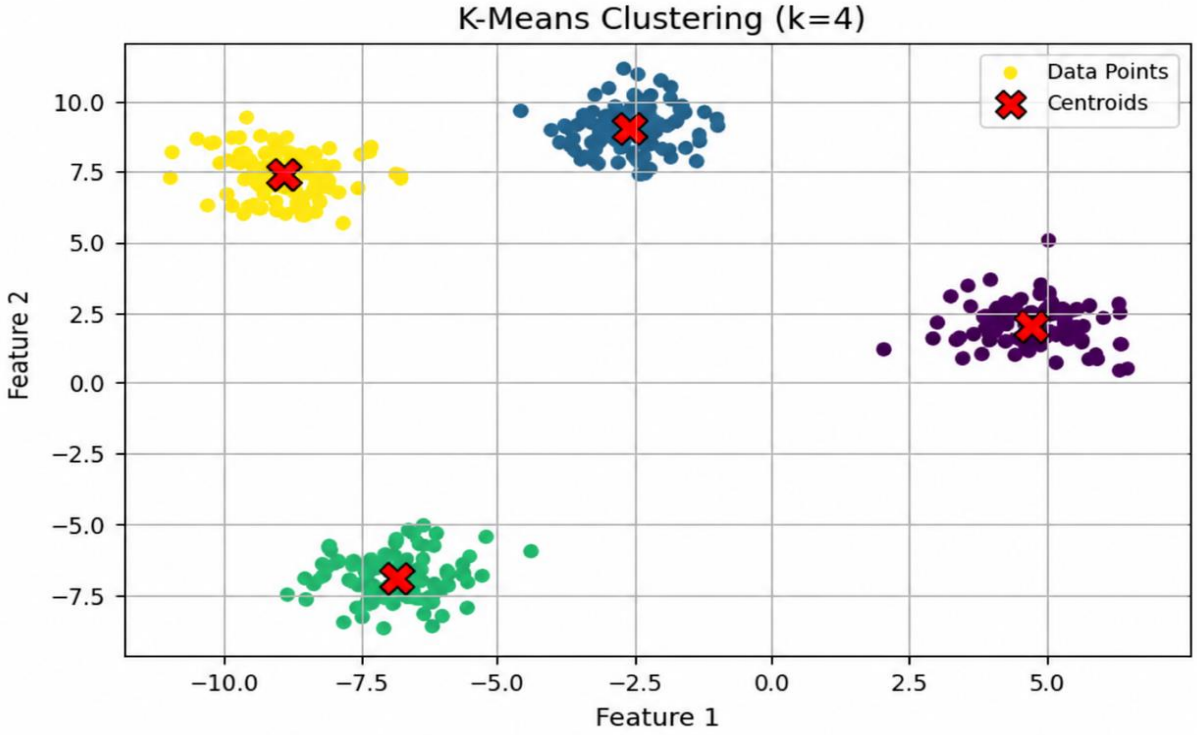
#3. Get cluster centers and labels
centroids = kmeans.cluster_centers_
labels = kmeans.labels_
print("--- K-Means Clustering Summary ---")
print(f"Number of clusters (k): {k}")
print("\nCluster Centroids:")
for i, center in enumerate(centroids):
    print(f"Cluster {i + 1} Center: [{center[0]:.4f}, {center[1]:.4f}]")

#4. Visualize the clusters and centroids
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=30, cmap='viridis', label='Data Points')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, marker='X',
    edgecolors='black', label='Centroids')
plt.title(f"K-Means Clustering (k={k})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

Output:

```
264928_VINAY SANJAY PANCHAL
--- K-Means Clustering Summary ---
Number of clusters (k): 4

Cluster Centroids:
Cluster 1 Center: [4.7257, 2.0031]
Cluster 2 Center: [-2.6084, 9.0377]
Cluster 3 Center: [-6.8830, -6.9632]
Cluster 4 Center: [-8.9110, 7.3703]
```



Theory:

K-Means Clustering is an **unsupervised machine learning algorithm** used to divide a dataset into a predefined number of groups or clusters. Unlike supervised learning, K-Means does not require labelled output data. It identifies patterns and groups similar data points together based on their distance from the cluster centers. It is widely used for customer segmentation, image compression, pattern recognition, and exploratory data analysis.

The main objective of K-Means clustering is to divide the data into **K clusters** in such a way that data points within the same cluster are as similar as possible, while data points belonging to different clusters are as different as possible. The algorithm uses a **centroid**, which represents the center of each cluster.

The K-Means algorithm generally works in the following steps: first, the number of clusters **K** is selected. Initial centroids are then chosen, usually randomly. Each data point is assigned to the

nearest centroid based on a distance measure, commonly **Euclidean distance**. After assigning the points, the centroids are recalculated as the mean of all points belonging to each cluster. The assignment and centroid calculation steps are repeated until the centroids become stable or the algorithm reaches the specified stopping condition.

The objective of K-Means can be represented using the **Within-Cluster Sum of Squares (WCSS)**, which measures the total squared distance between data points and their corresponding cluster centroids. The algorithm attempts to minimize this value. A lower WCSS generally indicates that the data points are located closer to their respective cluster centers.

In this practical, a synthetic dataset containing **400 samples and 4 distinct clusters** is generated using the **make_blobs()** function from Scikit-learn. The value of **K is set to 4**, meaning that the algorithm is instructed to divide the data into four clusters. The **KMeans()** class is used to initialize and train the clustering model. The parameter n_clusters=4 specifies the number of clusters, while random_state=42 ensures reproducible results. The n_init=10 parameter allows the algorithm to run with multiple initial centroid configurations and select a suitable result.

After fitting the model, the **cluster centroids** and **cluster labels** are obtained. The cluster_centers_ attribute provides the coordinates of the centroids, while the labels_ attribute provides the cluster assigned to each data point. The centroids represent the central location of the four clusters.

K-Means is simple, efficient, and suitable for large datasets. However, it requires the number of clusters to be specified in advance and can be affected by outliers and the initial selection of centroids. It also works best when clusters are relatively compact and well separated.

Thus, K-Means Clustering is an important unsupervised learning technique for discovering groups and patterns in data. In this practical, K-Means is implemented with **K = 4** to divide a synthetic dataset into four clusters and visualize their corresponding centroids.

Practical No: 9A

Aim: Write a Python script for Implementing Principle Component Analysis.

Code:

```
print("264928_VINAY SANJAY PANCHAL")
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. Generate synthetic dataset with 4 features
X, y = make_classification(
    n_samples=300,
    n_features=4,
    n_informative=3,
    n_redundant=1,
    n_classes=3,
    random_state=42
)

# 2. Standardize features (mean=0, variance=1)
X_scaled = StandardScaler().fit_transform(X)

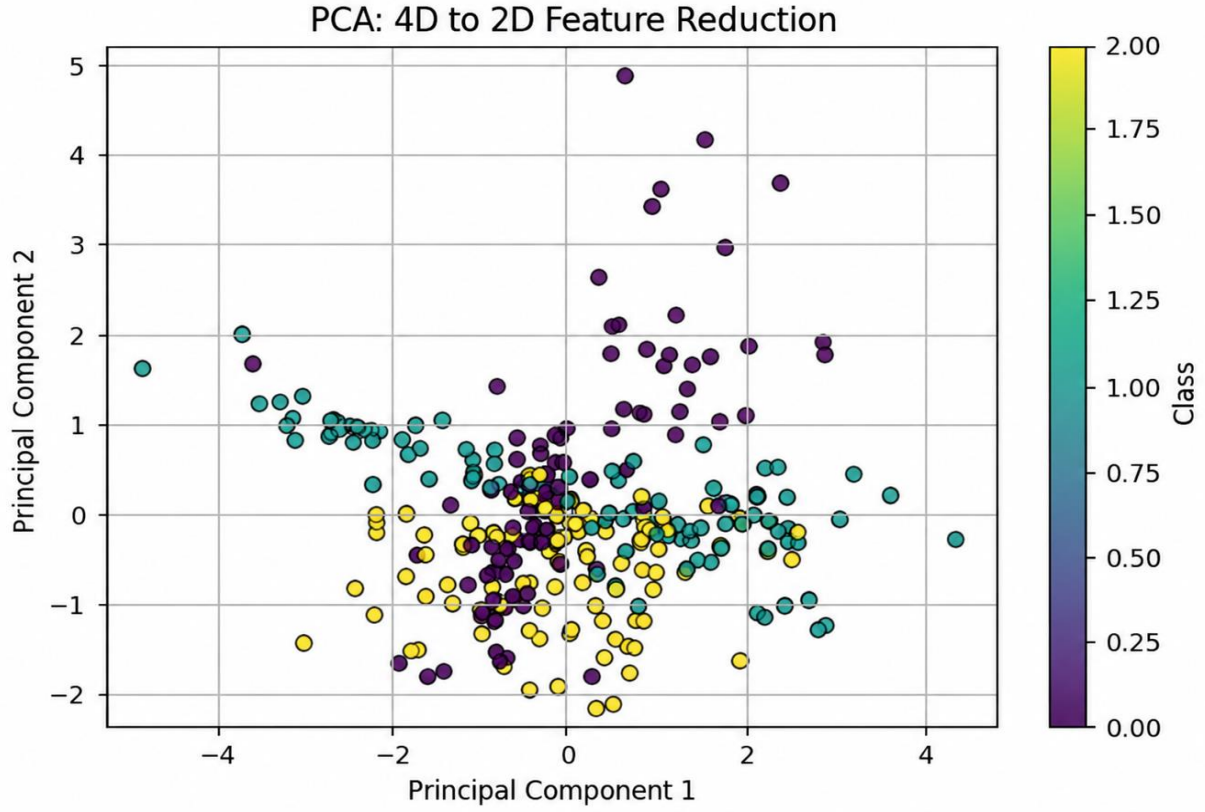
# 3. Initialize PCA to reduce data from 4D to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

#4. Print Explained Variance
print("--- Principal Component Analysis (PCA) ---")
print(f"Explained Variance Ratio by Component: {pca.explained_variance_ratio_}")
print(f"Total Explained Variance (2 Components): {sum(pca.explained_variance_ratio_) * 100:.2f}%")

#5. Visualize the reduced 2D dataset
plt.figure(figsize=(8, 5))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolors='k', alpha=0.8)
plt.title("PCA: 4D to 2D Feature Reduction")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(scatter, label="Class")
plt.grid(True)
plt.show()
```

Output:

```
264928_VINAY SANJAY PANCHAL
--- Principal Component Analysis (PCA) ---
Explained Variance Ratio by Component: [0.55183421 0.26762724]
Total Explained Variance (2 Components): 81.95%
```



Theory:

Principal Component Analysis (PCA) is a dimensionality reduction technique used to reduce the number of features in a dataset while preserving as much important information as possible. In datasets with a large number of features, working with all the features can increase computational complexity and make data visualization difficult. PCA transforms the original correlated features into a smaller set of new, uncorrelated variables called **Principal Components**.

The principal components are arranged according to the amount of variance they explain in the data. The **first principal component (PC1)** captures the maximum possible variance, while the second principal component (PC2) captures the maximum remaining variance subject to being uncorrelated with the first component. Similarly, additional components explain the remaining variance in decreasing order.

Before applying PCA, it is generally important to **standardize the features**, especially when the features have different scales or units. Standardization transforms the features so that they have a mean of approximately 0 and a variance of 1. In this practical, the **StandardScaler()** function from Scikit-learn is used to standardize the generated dataset.

In this practical, a synthetic dataset containing **300 samples and 4 features** is generated using the **make_classification()** function. The dataset contains three classes. The features are standardized using **StandardScaler()** before applying PCA. This ensures that features with larger numerical values do not dominate the principal components.

The PCA transformation is performed using the PCA(n_components=2) class. The parameter n_components=2 specifies that the original four-dimensional dataset should be reduced to two dimensions. The **fit_transform()** function learns the principal components from the standardized data and transforms the original data into the new two-dimensional representation.

One of the important outputs of PCA is the **Explained Variance Ratio**. It indicates the proportion of the total variance in the original dataset that is represented by each principal component. The sum of the explained variance ratios of the selected components gives the total amount of information retained after dimensionality reduction. For example, if two components explain 80% of the variance, the two-dimensional representation retains approximately 80% of the variation present in the original data.

The reduced dataset is visualized using a scatter plot. The first principal component is represented on the X-axis and the second principal component on the Y-axis. Different classes are represented using different colours. This makes it possible to observe the distribution and separation of the classes in the reduced two-dimensional space.

Thus, Principal Component Analysis is an important technique for reducing the dimensionality of datasets while retaining the maximum possible information. In this practical, PCA is used to reduce a **4-dimensional dataset to 2 dimensions**, calculate the explained variance, and visualize the transformed data.

Practical No: 9B

Aim: Write a Python script for Implementing Support Vector Machine.

Code:

```
#9B Implement SVM.
print("264928_VINAY SANJAY PANCHAL")
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

svm = SVC()
param_grid = {
   'C': [0.1, 1, 10, 100],
   'gamma': [1, 0.1, 0.01, 0.001],
   'kernel': ['rbf', 'linear']
}
grid = GridSearchCV(svm, param_grid, cv=5, refit=True, verbose=0)
grid.fit(X_train, y_train)
print(f"Best Parameters: {grid.best_params_}")
y_pred = grid.predict(X_test)

print("\n--- Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred): .4f}")
print("\nClassification Report: \n", classification_report(y_test, y_pred, target_names=iris.target_names))
print("\n--- Confusion Matrix ---")
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, cmap='YlGnBu', xticklabels=iris.target_names, yticklabels=iris.target_names, fmt='d'
plt.xlabel("Predicted")
plt.show()
```

Output:

```
264928_VINAY SANJAY PANCHAL
Best Parameters: {'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}

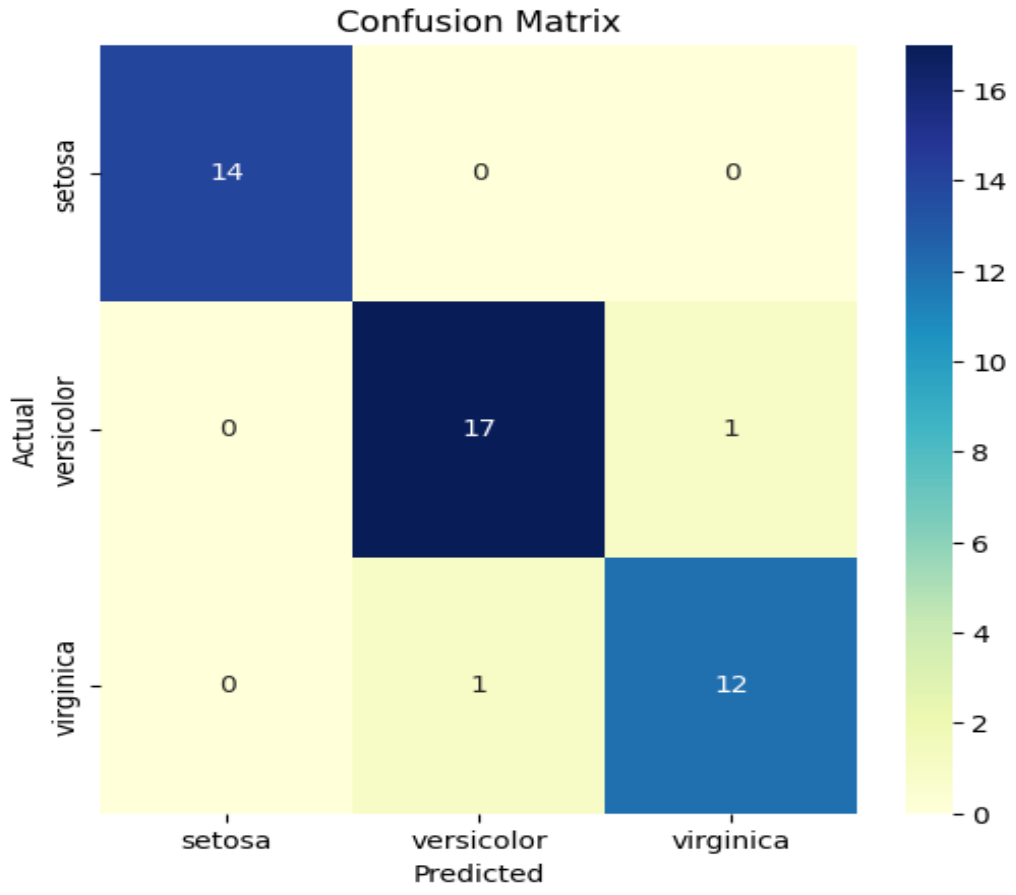
--- Model Evaluation ---
Accuracy:  0.9556

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       0.94      0.94      0.94        18
   virginica       0.92      0.92      0.92        13

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45
```

```
--- Confusion Matrix ---
```



Theory:

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression problems. It is particularly effective for classification tasks where the goal is to find a suitable boundary that separates different classes of data. SVM attempts to find the best decision boundary, known as a **hyperplane**, that separates the classes while maximizing the distance between the hyperplane and the nearest data points from each class.

The data points closest to the decision boundary are called **Support Vectors**. These points are important because they determine the position and orientation of the optimal hyperplane. The margin is the distance between the decision boundary and the nearest support vectors. SVM attempts to maximize this margin because a larger margin can provide better generalization to unseen data.

SVM can handle both linearly and non-linearly separable data. For non-linear classification, SVM uses a technique called the **Kernel Trick**. Common kernel functions include Linear, Polynomial, and Radial Basis Function (RBF) kernels. The RBF kernel is useful when the relationship between features and classes is non-linear.

In this practical, the **Iris dataset** from Scikit-learn is used. The Iris dataset contains 150 samples with four features: sepal length, sepal width, petal length, and petal width. It contains three classes of iris flowers: setosa, versicolor, and virginica. The dataset is divided into training and testing sets using **train_test_split()**. In this program, 70% of the data is used for training and 30% for testing.

The **SVC()** class is used to create the Support Vector Classifier. In this practical, **GridSearchCV** is used to find the best combination of hyperparameters. The parameters tested include **C**, **gamma**, and **kernel**. The parameter C controls the trade-off between achieving a large margin and correctly classifying training data. The gamma parameter controls the influence of individual training points when using kernels such as RBF. The kernel determines the type of decision boundary used by the model.

GridSearchCV performs **5-fold cross-validation** for each combination of parameters and selects the combination that provides the best performance. The selected model is then used to predict the classes of the test data.

The model is evaluated using **Accuracy, Classification Report, and Confusion Matrix**. Accuracy represents the proportion of correctly classified test samples. The classification report provides precision, recall, and F1-score for each Iris class. The confusion matrix shows the number of correct and incorrect predictions for each class and helps identify which classes are being confused by the model.

Thus, Support Vector Machine is an important supervised learning algorithm for classification. In this practical, SVM is implemented on the Iris dataset with feature standardization and GridSearchCV to select suitable hyperparameters. The trained model is then evaluated using accuracy, classification report, and confusion matrix.